Prédiction intelligente du handover dans les réseaux LTE

Cours complet — télécommunications mobiles et apprentissage automatique, construit à partir du projet *Intelligent-Handover-Prediction-LTE*.

Chapitre 1 — Réseaux mobiles et architecture LTE

> **Objectif du chapitre :** savoir expliquer, schéma à l'appui, comment est construit un réseau LTE, qui fait quoi, et où se situe le *handover* dans cette architecture.

1.1 Pourquoi un réseau « cellulaire » ?

Une antenne ne peut pas couvrir un pays entier : la puissance reçue décroît très vite avec la distance, et le spectre radio (les fréquences) est une ressource rare et chère. La solution, depuis la 1G, est de **découper le territoire en cellules** : chaque station de base couvre une petite zone et **réutilise les mêmes fréquences** que des cellules plus éloignées.

Cela crée immédiatement deux problèmes, qui sont au cœur de ce projet :

1. **L'interférence** : une cellule voisine qui émet sur la même fréquence perturbe le signal utile. C'est pour cela qu'on parle de **SINR** (rapport signal / interférence + bruit) et pas seulement de puissance reçue.
2. **La mobilité** : un utilisateur qui se déplace sort de sa cellule et doit être **transféré** vers une autre, sans couper sa communication. C'est le **handover** (en français : *transfert intercellulaire*).

1.2 Des générations successives (2G → 5G)

Génération	Technologie principale	Ce qui change	Accès radio
2G	GSM (1991)	voix numérique, SMS	TDMA/FDMA
3G	UMTS / HSPA (2001)	données mobiles, Internet	WCDMA
4G	**LTE / LTE-Advanced (2009)**	**tout-IP, haut débit, faible latence**	**OFDMA (descendant), SC-FDMA (montant)**
5G	NR (2019)	très haut débit, IoT massif, ultra-faible latence	OFDMA flexible (numérologies)

LTE (*Long Term Evolution*, normes 3GPP à partir de la *Release 8*) est la technologie de ce projet. Deux idées le distinguent de la 3G :

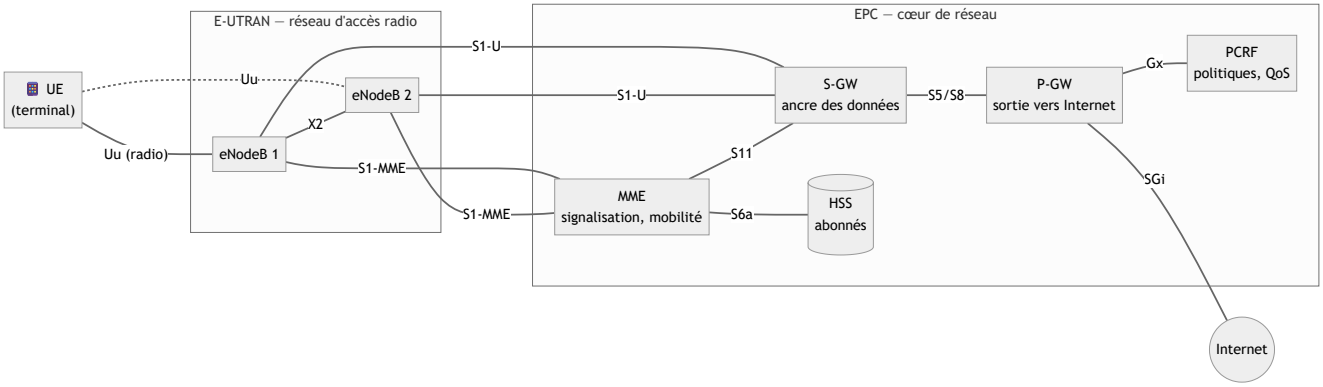
- **tout-IP** : il n'y a plus de réseau « circuit » pour la voix ; tout est paquet IP (la voix passe en VoLTE) ;
- **OFDMA** : la bande est découpée en milliers de sous-porteuses étroites (15 kHz), ce qui résiste bien aux trajets multiples et permet d'allouer finement les ressources (chapitre 2).

> 💡 **Pourquoi LTE et pas 5G pour ce projet ?** LTE est massivement déployé, ses procédures de mesure et de handover (TS 36.331) sont stables et bien documentées, et la 5G NR reprend **les mêmes principes** (mesures RSRP/RSRQ/SINR, événements A3/A5, TTT). Tout ce qui est appris ici se transpose à la 5G.

1.3 L'architecture LTE (EPS)

Le système complet s'appelle **EPS** (*Evolved Packet System*). Il a deux grandes parties :

- **E-UTRAN** (le réseau d'accès radio) : uniquement des **eNodeB** (les stations de base) ;
- **EPC** (*Evolved Packet Core*, le cœur de réseau) : MME, S-GW, P-GW, HSS, PCRF.



Rôle de chaque élément

Élément	Rôle	Analogie
UE (*User Equipment*)	le terminal ; il **mesure** la qualité radio des cellules et la rapporte	le passager
eNodeB	station de base : gère la radio, **ordonnance** les ressources, **décide les handovers**	le contrôleur local
MME (*Mobility Management Entity*)	plan de contrôle : authentification, suivi de localisation, établissement des sessions, signalisation des handovers S1	le centre de régulation
S-GW (*Serving Gateway*)	achemine les paquets de l'utilisateur ; **ancre de mobilité** entre eNodeB	l'aiguillage
P-GW (*PDN Gateway*)	sortie vers Internet, attribution de l'adresse IP, facturation	la frontière
HSS (*Home Subscriber Server*)	base de données des abonnés (clés, profils)	l'annuaire
PCRF	règles de qualité de service et de facturation	le règlement

Les interfaces à connaître

- **Uu** : l'interface radio entre UE et eNodeB (c'est là que se font les **mesures**).
- **X2** : liaison **directe entre deux eNodeB**. Elle sert au **handover X2** (le plus courant) et à la coordination d'interférences.
- **S1** : entre eNodeB et cœur. **S1-MME** pour la signalisation (plan de contrôle), **S1-U** pour les données (plan utilisateur).

> 🎯 **Point clé pour l'entretien :** en LTE, **c'est l'eNodeB source qui décide du handover**, sur la base des **rapports de mesure** envoyés par l'UE. Il n'y a plus de contrôleur centralisé comme le RNC de la 3G : l'architecture est « plate ». C'est exactement pour cela qu'un modèle prédictif déployé **au niveau de l'eNodeB** (ou d'un contrôleur SON) est pertinent.

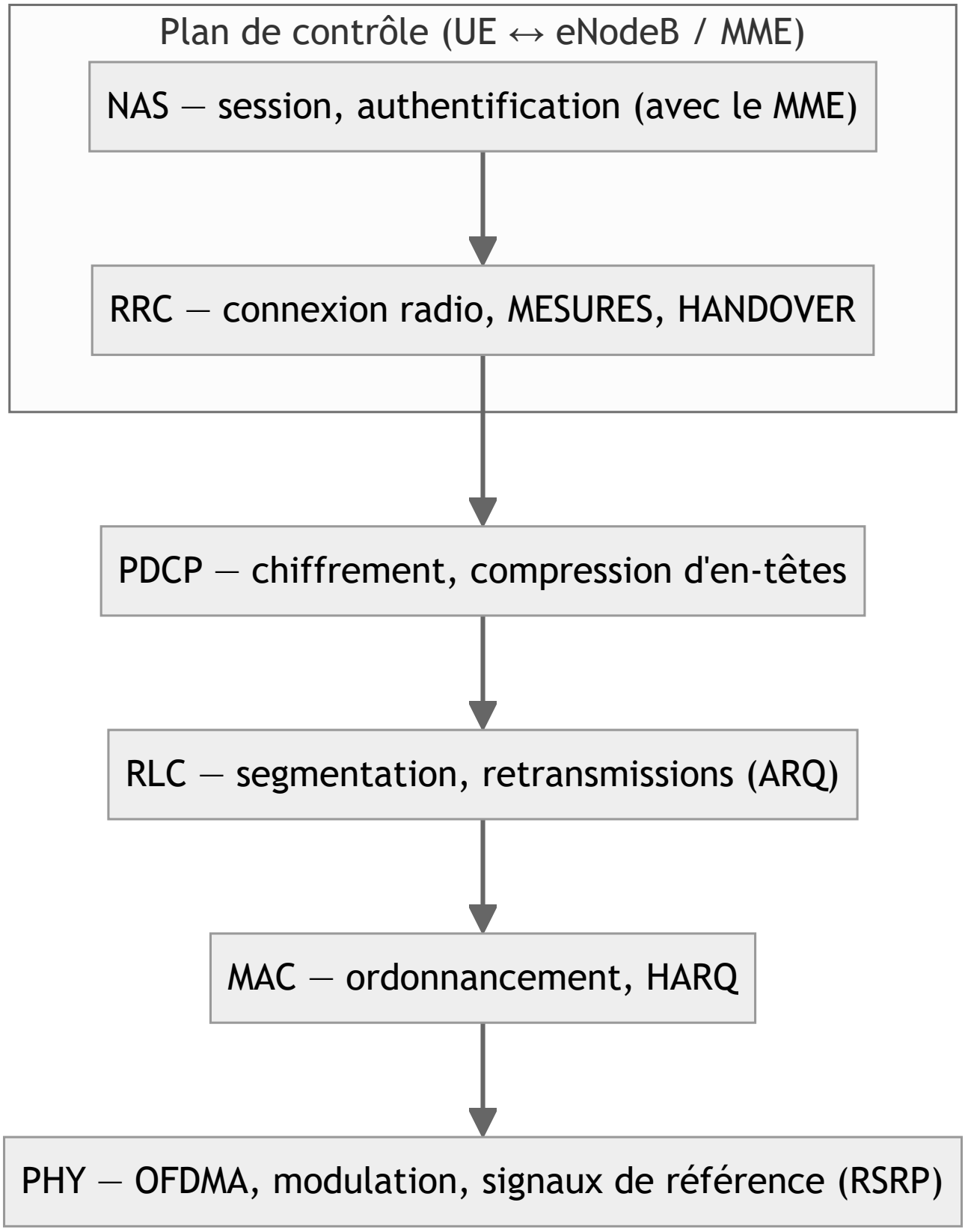
1.4 Plan de contrôle et plan utilisateur

Tout trafic dans le réseau appartient à l'un de deux plans :

- **Plan utilisateur (U-plane)** : les données de l'utilisateur (vidéo, web…). Chemin : UE → eNodeB → S-GW → P-GW → Internet.
- **Plan de contrôle (C-plane)** : la **signalisation** qui pilote le réseau (connexion, mesures, handover). Chemin : UE → eNodeB (RRC) → MME (NAS).

Le handover est avant tout une **procédure du plan de contrôle** (messages RRC et X2AP), qui a des **conséquences sur le plan utilisateur** (les paquets doivent être redirigés vers la nouvelle cellule sans perte).

1.5 La pile protocolaire radio


Plan de contrôle (UE ↔ eNodeB / MME)
NAS — session, authentification (avec le MME)
RRC — connexion radio, MESURES, HANDOVER
PDCP — chiffrement, compression d'en-têtes
RLC — segmentation, retransmissions (ARQ)
MAC — ordonnancement, HARQ
PHY — OFDMA, modulation, signaux de référence (RSRP)

Couche	Ce qu'elle fait	Lien avec le projet
PHY (couche 1)	transmission OFDMA, signaux de référence, mesures physiques	le **RSRP** est mesuré ici sur les signaux de référence
MAC	ordonnancement des ressources, HARQ	utilise le **CQI** pour choisir la modulation
RLC / PDCP	fiabilité, chiffrement	transfert des paquets en cours lors d'un handover
RRC (couche 3)	gestion de la connexion radio	**configure les mesures**, reçoit les **rapports**, envoie l'**ordre de handover** ; applique le **filtrage L3**

> 💡 **« L1 » et « L3 »** dans le code (`l3_rsrp_serving`) font référence à ces couches : la mesure brute est faite en couche 1 (PHY), puis **lissée par un filtre en couche 3** (RRC) avant d'être comparée aux seuils de handover.

1.6 Les états RRC : où a lieu le handover ?

Un UE est toujours dans l'un de deux états :

État	Situation	Mobilité
RRC_IDLE	pas de communication active (téléphone en veille)	**resélection de cellule** décidée par l'UE lui-même, sans signalisation
RRC_CONNECTED	communication active	**handover** décidé par le réseau (eNodeB), sur la base des mesures de l'UE

Ce projet traite le handover en RRC_CONNECTED : c'est là qu'un handover raté coupe une communication (perte d'appel, coupure de vidéo) et qu'une prédiction a de la valeur.

1.7 Ce qu'il faut retenir

- Un réseau cellulaire réutilise les fréquences → **interférence** et **mobilité** sont les deux problèmes fondamentaux.
- LTE = **E-UTRAN** (eNodeB) + **EPC** (MME, S-GW, P-GW, HSS, PCRF) ; architecture **plate**, tout-IP.
- Les eNodeB communiquent directement par **X2** ; c'est le chemin du handover le plus courant.
- **L'eNodeB source décide le handover** à partir des **rapports de mesure RRC** de l'UE.
- Le projet se place en **RRC_CONNECTED**, sur la mobilité **intra-LTE, intra-fréquence**.

Chapitre 2 — Couche physique et propagation radio

> **Objectif :** comprendre d'où viennent les nombres que le modèle reçoit en entrée. Pourquoi le signal varie, à quelles échelles, et comment le simulateur reproduit ces variations.

2.1 Les décibels : l'outil indispensable

En radio, les puissances couvrent des dizaines d'ordres de grandeur (de 40 W à l'antenne à 10^{-14} W au terminal). On les exprime donc en **échelle logarithmique** :

- **dB** : un *rapport* de puissances. $G_{dB} = 10 \log_{10}(P_1/P_2)$. Exemple : ×2 ≈ +3 dB, ×10 = +10 dB, ×1000 = +30 dB.
- **dBm** : une *puissance absolue*, rapportée à 1 mW. $P_{dBm} = 10 \log_{10}(P/1\,\mathrm{mW})$. Exemple : 1 W = 30 dBm, 40 W ≈ 46 dBm, 1 pW = −90 dBm.

Règles à connaître par cœur :

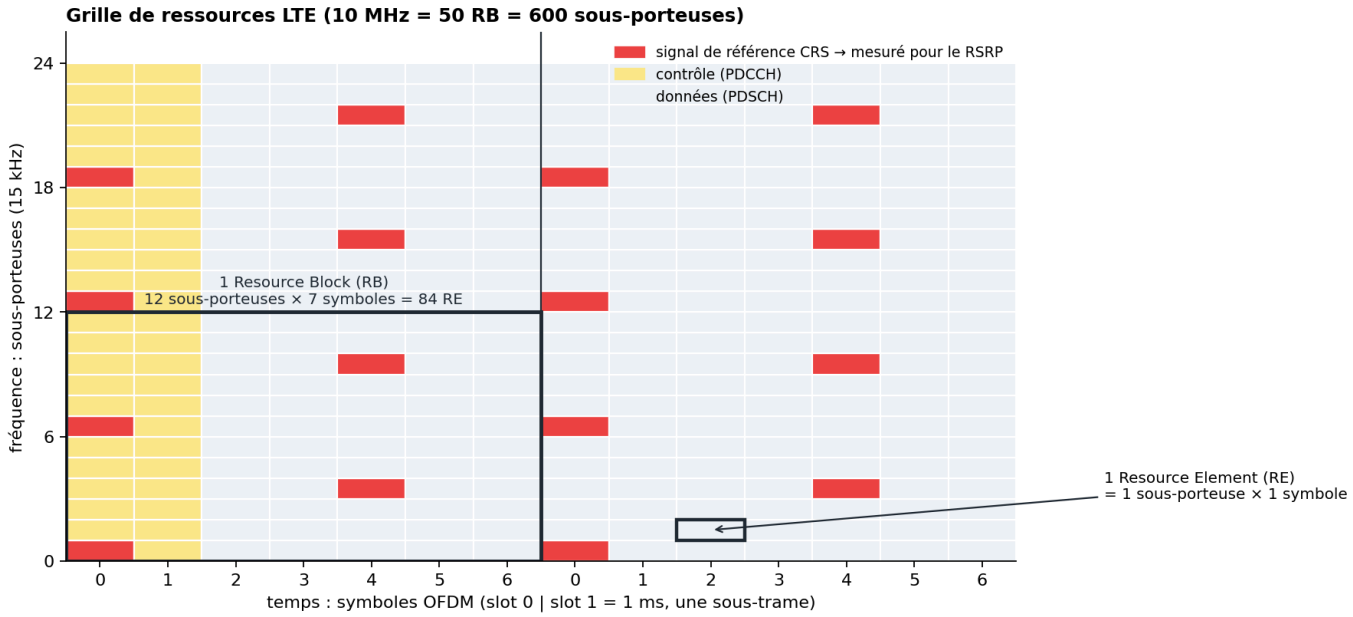
Opération en linéaire	Équivalent en dB
multiplier par un gain	**additionner** des dB
diviser (perte)	**soustraire** des dB
additionner deux puissances	**pas** une addition en dB : il faut repasser en linéaire (mW), sommer, revenir en dB

> ⚠️ Le troisième point est un piège classique : deux signaux de −100 dBm donnent −97 dBm, pas −200 dBm. Le code du simulateur repasse toujours en mW avant de sommer (`compute_sinr` , `compute_rsrq`).

2.2 OFDMA et la grille de ressources

LTE découpe la bande en **sous-porteuses** de 15 kHz, orthogonales entre elles (OFDM). Dans le temps, le signal est découpé en **symboles OFDM** (≈ 71 µs). La plus petite unité est donc une case de cette grille :

- **Resource Element (RE)** : 1 sous-porteuse × 1 symbole ;
- **Resource Block (RB)** : 12 sous-porteuses (180 kHz) × 7 symboles (un *slot* de 0,5 ms), soit 84 RE. C'est l'unité que l'ordonnanceur attribue aux utilisateurs.



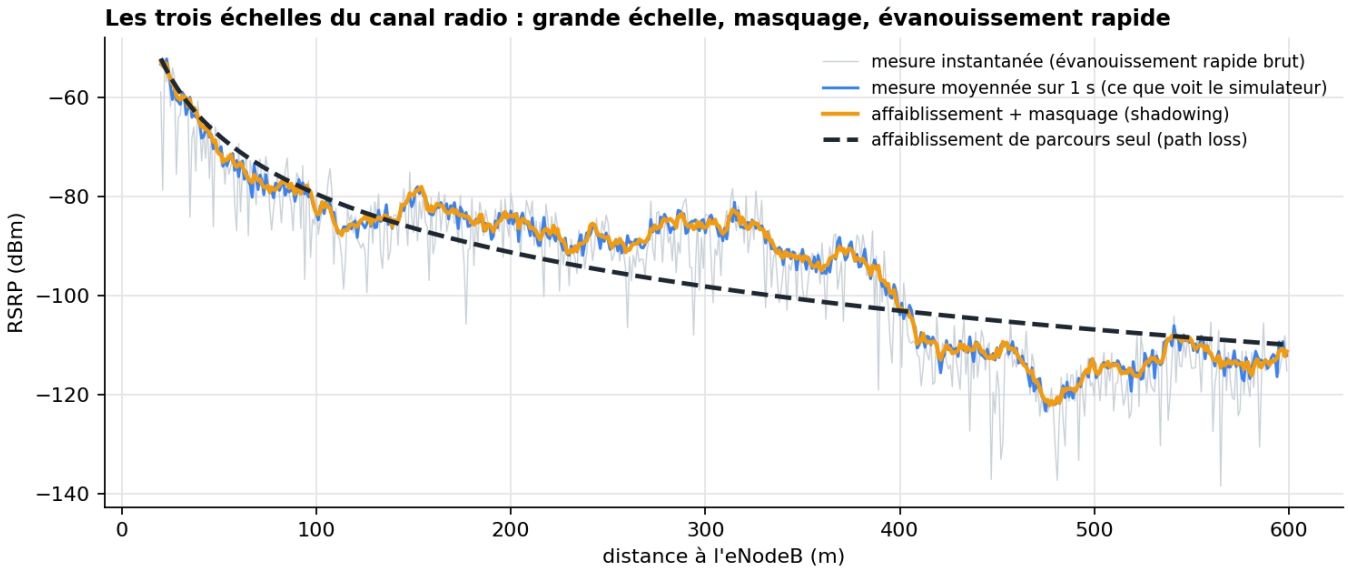
Largeur de bande	Nombre de RB	Sous-porteuses utiles
1,4 MHz	6	72
5 MHz	25	300
10 MHz (projet)	**50**	**600**
20 MHz	100	1200

Parmi ces RE, certains portent des **signaux de référence** (*Cell-specific Reference Signals*, CRS), en rouge ci-dessus. Leur position et leur puissance sont connues à l'avance : le terminal les utilise pour **estimer le canal** et **mesurer la qualité de chaque cellule**. C'est sur ces RE précis qu'est défini le **RSRP** (chapitre 3).

> 🎯 **Conséquence directe dans le projet :** l'eNodeB émet 46 dBm (≈ 40 W) **au total**, répartis sur 600 sous-porteuses. Un seul RE reçoit donc $46 - 10 \cdot \log_{10}(600) \approx$ **18,2 dBm**. Le RSRP doit être calculé à partir de cette puissance *par RE*. La version 3 du simulateur utilisait les 46 dBm entiers : tous les RSRP étaient trop élevés de 27,8 dB, et 20 % des valeurs dépassaient le maximum reportable de −44 dBm. La v4 corrige cela (`RadioConfig.rs_power_dbm`).

2.3 Les trois échelles du canal radio

Le signal reçu par un terminal varie pour trois raisons physiques différentes, à trois échelles différentes :



Phénomène	Cause	Échelle spatiale	Amplitude typique	Dans le simulateur
Affaiblissement de parcours (*path loss*)	la puissance se dilue avec la distance	des centaines de m	60 à 140 dB	`path_loss_los` / `path_loss_nlos`
Masquage (*shadowing*)	bâtiments, relief qui masquent le trajet	~50 à 100 m	σ = 4 à 8 dB	`update_shadow` (Gudmundson)
Évanouissement rapide (*fast fading*)	trajets multiples qui s'additionnent ou s'annulent	λ/2 ≈ 7,5 cm à 2 GHz	jusqu'à −30 dB instantané	`update_fast_fading` (moyenné sur 1 s)

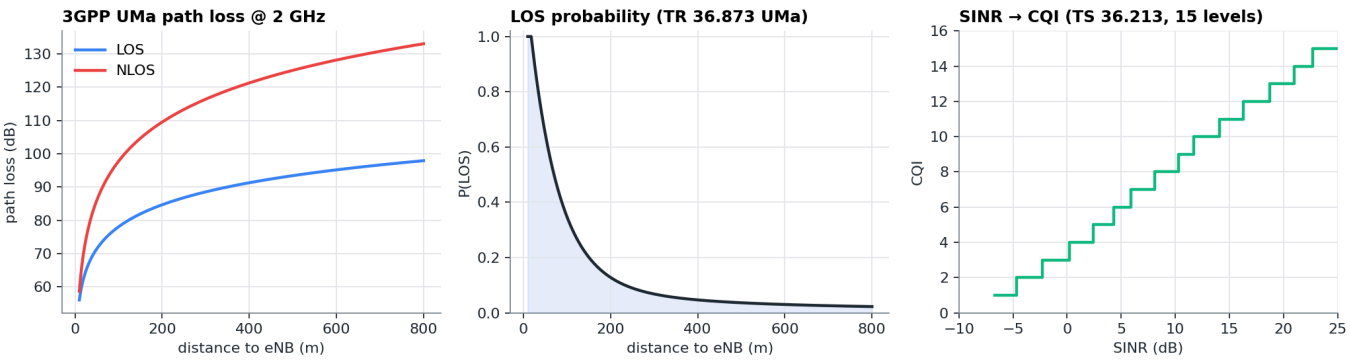
a) Affaiblissement de parcours : le modèle 3GPP UMa

En espace libre (formule de Friis), la puissance décroît en $1/d^2$, soit −20 dB par décade de distance. En ville, avec les réflexions sur le sol et les bâtiments, la décroissance est plus rapide, typiquement $1/d^{3,5}$ à $1/d^4$.

Le projet utilise le modèle **3GPP TR 36.873 UMa** (*Urban Macro* : antennes sur les toits, ville dense), avec deux formules selon que le trajet direct existe ou non :

$$PL_{LOS} = 22\log_{10}(d) + 28 + 20\log_{10}(f_c) \qquad PL_{NLOS} = \max\left(PL_{LOS},\ 19{,}55 + 39{,}09\log_{10}(d)\right)$$

avec d en mètres et f_c en GHz (2 GHz ici). La pente NLOS (39 dB/décade) est presque le double de la pente LOS (22 dB/décade).



b) LOS ou NLOS ? Une probabilité qui dépend de la distance

Le trajet direct (LOS, *line of sight*) existe-t-il entre l'antenne et le terminal ? Près de l'antenne, souvent ; loin, rarement. 3GPP donne :

$$P(LOS \mid d) = \min\left(\frac{18}{d}, 1\right)\left(1 - e^{-d/63}\right) + e^{-d/63}$$

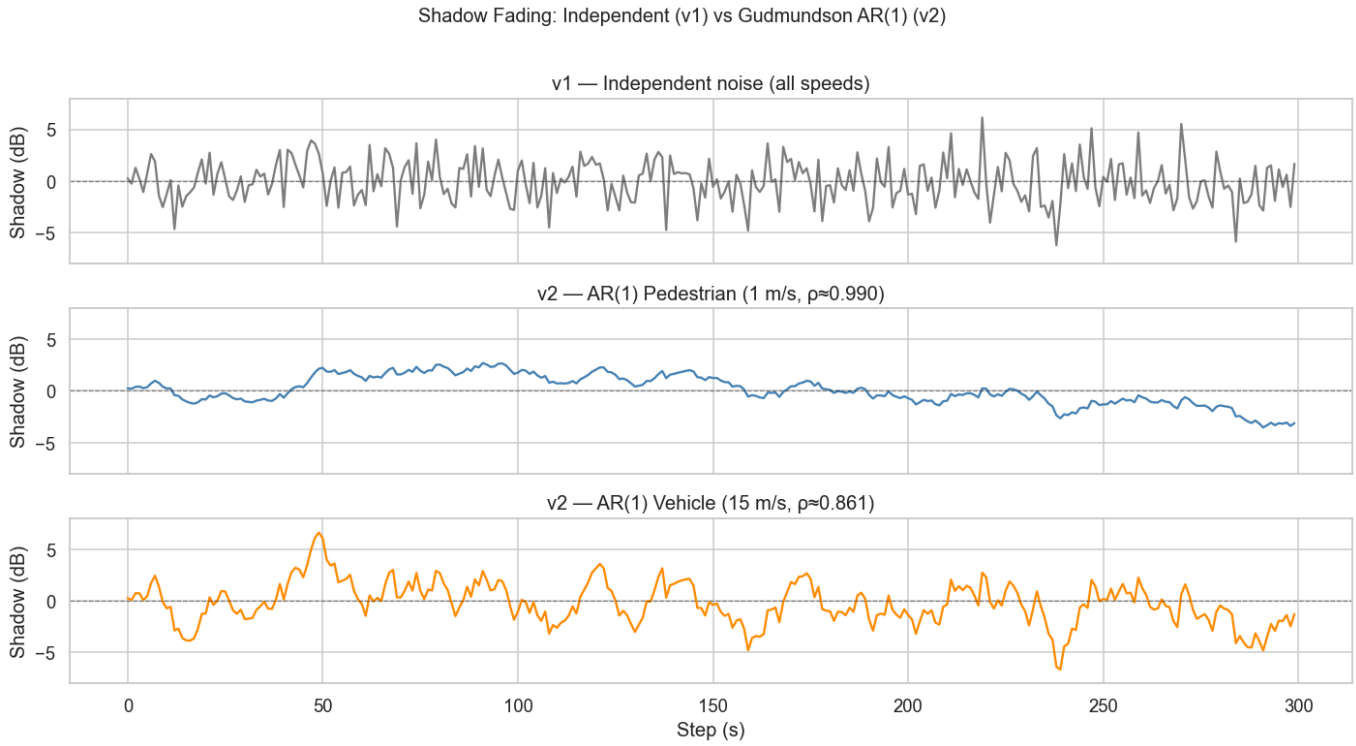
Soit environ 65 % à 50 m, 35 % à 100 m, 13 % à 200 m, 5 % à 400 m.

Cohérence spatiale (v4). Il ne suffit pas de tirer LOS/NLOS au hasard : un piéton immobile ne doit pas basculer. La v3 retirait l'état toutes les 5 secondes, indépendamment du déplacement, ce qui provoquait des sauts de ~20 dB même pour un terminal à l'arrêt, et des handovers en cascade. La v4 suit la procédure de cohérence spatiale de **TR 38.901** : un processus gaussien latent z évolue avec la distance parcourue ($\rho = e^{-\Delta d/50\,\mathrm{m}}$), et le lien est LOS si $\Phi(z) < P(LOS \mid d)$. La probabilité marginale reste exacte, et l'état ne change que si l'UE bouge.

c) Masquage (shadowing) : le modèle de Gudmundson

Le masquage suit une loi **log-normale** : en dB, c'est une gaussienne d'écart-type σ (4 dB en LOS, 6 dB en NLOS). Il est **corrélé dans l'espace** : deux points distants de quelques mètres ont presque le même masquage. Le modèle de Gudmundson l'exprime comme un processus AR(1) sur la distance :

$$S_n = \rho\, S_{n-1} + \sqrt{1-\rho^2}\,\mathcal{N}(0,\sigma^2), \qquad \rho = e^{-\Delta d/d_{corr}},\ d_{corr} = 100\ \mathrm{m}$$



Sortie du notebook 06. En haut, un masquage tiré indépendamment à chaque seconde (irréaliste : le terrain ne change pas d'une seconde à l'autre). Au milieu et en bas, le modèle de Gudmundson : pour un piéton (1 m/s) la corrélation d'un pas à l'autre est ρ ≈ 0,99 et le masquage dérive lentement ; pour un véhicule (15 m/s), ρ ≈ 0,86 et il varie plus vite.

d) Évanouissement rapide : Rayleigh, Rice et moyennage

Le signal arrive par de nombreux trajets (réflexions, diffractions) qui s'additionnent avec des phases différentes. Sans trajet direct, l'amplitude suit une loi de **Rayleigh** ; avec un trajet direct dominant, une loi de **Rice** (facteur K = 5 dB dans le projet). Ces évanouissements varient **toutes les demi-longueurs d'onde** : à 2 GHz, λ = c/f = 15 cm, donc tous les 7,5 cm.

Un terminal ne rapporte pas une valeur instantanée : il **moyenne** sur une période de mesure. Sur 1 s, un piéton à 1 m/s traverse ~14 zones indépendantes et un véhicule à 15 m/s ~200. La moyenne de N échantillons indépendants a une dispersion réduite d'un facteur ~√N. C'est ce que fait la v4 : environ 1,2 dB d'écart-type pour un piéton, 0,3 dB pour un véhicule, au lieu des ±5 dB d'un échantillon unique.

> 💡 **Pourquoi c'est important pour le ML :** si l'on simule mal ces échelles, le modèle apprend des artefacts. Avec un évanouissement non moyenné, il apprendrait à « prédire » des handovers déclenchés par du bruit, ce qui n'existe pas dans un vrai réseau.

2.4 Bruit, interférence et SINR

Le signal utile n'est pas seul : il y a du **bruit thermique** et l'**interférence** des cellules voisines qui réutilisent la même fréquence.

Bruit thermique. $N = kTB$: −174 dBm/Hz à température ambiante. Sur les 9 MHz réellement occupés par 50 RB, cela fait −174 + 10·log₁₀(9·10⁶) ≈ −104,5 dBm. On ajoute le **facteur de bruit** du récepteur (~9 dB pour un terminal), soit **N ≈ −95,5 dBm**.

Interférence. Chaque cellule voisine k contribue en proportion de sa charge : une cellule vide n'émet que ses signaux de référence (~10 % du temps-fréquence), une cellule pleine émet partout.

$$SINR = \frac{S}{\sum_k P_k \cdot \left(0{,}1 + 0{,}9 \cdot \text{charge}_k\right) + N}$$

Dans le projet (4 cellules espacées de 500 m), le réseau est **limité par l'interférence** : l'interférence dépasse largement le bruit presque partout. Le SINR est donc surtout déterminé par le **rapport entre la cellule de service et les voisines**, ce qui en fait un excellent indicateur de proximité de la frontière de cellule.

> 🎯 **Question classique :** « Pourquoi le SINR baisse-t-il avant un handover ? » Parce qu'en approchant de la frontière, le signal de la cellule de service baisse **et** celui de la voisine (qui est de l'interférence !) monte. Le SINR chute donc doublement. C'est l'une des raisons pour lesquelles un handover tardif échoue souvent : au moment de l'exécution, le SINR est trop faible pour que les messages de signalisation passent.

2.5 Ce qu'il faut retenir

- Les puissances s'additionnent en **mW**, pas en dB.
- LTE 10 MHz = **50 RB = 600 sous-porteuses** ; le RSRP se mesure **par RE**, sur les signaux de référence.

- Trois échelles de variation : **path loss** (distance), **shadowing** (obstacles, ~100 m), **fast fading** (λ/2, moyenné par la mesure).
- LOS/NLOS selon 3GPP, avec **cohérence spatiale** (v4).
- SINR = signal / (interférence pondérée par la charge + bruit). Le réseau du projet est **limité par l'interférence**.

Chapitre 3 — Les indicateurs radio (KPI) : RSRP, RSRQ, SINR, CQI

> **Objectif :** savoir définir précisément chaque KPI utilisé par le modèle, donner ses ordres de grandeur, et surtout **justifier pourquoi ce sont ces indicateurs-là qui servent à prédire un handover**, et pas d'autres.

3.1 Vue d'ensemble

KPI	Ce qu'il mesure	Unité / plage	Qui le mesure	Colonne du dataset
RSRP	la **puissance** reçue d'une cellule	dBm, −140 à −44	l'UE, sur les signaux de référence	`rsrp_serving`, `rsrp_neighbor`
RSRQ	la **qualité** : part de la cellule dans tout ce qui est reçu	dB, −19,5 à −3	l'UE	`rsrq_serving`, `rsrq_neighbor`
SINR	signal / (interférence + bruit)	dB, ≈ −20 à +30	l'UE (non standardisé en LTE)	`sinr`
CQI	qualité de canal **résumée en un indice** pour l'ordonnanceur	1 à 15	l'UE, envoyé à l'eNodeB	`cqi`
RSRP filtré L3	RSRP lissé par la couche RRC	dBm	l'UE (filtre RRC)	`l3_rsrp_serving`, `l3_rsrp_neighbor`

3.2 RSRP — *Reference Signal Received Power*

Définition (3GPP TS 36.214). Moyenne linéaire de la puissance reçue sur les **Resource Elements qui portent les signaux de référence (CRS)** d'une cellule, dans la bande de mesure.

C'est donc la puissance **d'un seul RE**, et pas la puissance totale. Cela rend le RSRP **indépendant de la largeur de bande et de la charge** de la cellule : il ne dépend que de la **distance et du canal** entre l'UE et l'antenne. C'est exactement ce qu'on veut pour comparer des cellules entre elles.

Plage de rapport (TS 36.133) : de −140 dBm à −44 dBm, par pas de 1 dB (valeurs RSRP_00 à RSRP_97). Une valeur au-dessus de −44 dBm est rapportée « −44 ».

Ordres de grandeur (usage terrain courant) :

RSRP	Qualité
> −80 dBm	excellente (près de l'antenne)
−80 à −90 dBm	bonne
−90 à −100 dBm	moyenne
−100 à −110 dBm	faible (bord de cellule)
< −110 dBm	très faible, risque de perte de lien

Exemple chiffré du projet. eNodeB 46 dBm au total → 18,2 dBm par RE. À 300 m en NLOS : $PL = 19{,}55 + 39{,}09 \log_{10}(300) \approx 116{,}4$ dB → RSRP ≈ 18,2 − 116,4 ≈ **−98 dBm** avant masquage, typique d'une zone de bord de cellule en ville.

> 🎯 **Pourquoi le RSRP est le KPI n°1 du handover :** l'événement A3 (chapitre 4), qui déclenche la grande majorité des handovers, compare **le RSRP de la voisine à celui de la cellule de service**. Le modèle reçoit donc directement la grandeur qui pilote la décision.

3.3 RSRQ — *Reference Signal Received Quality*

Définition :

$$RSRQ = N \cdot \frac{RSRP}{RSSI}$$

- **RSSI** (*Received Signal Strength Indicator*) : **toute** la puissance reçue sur la bande (signal utile + interférence de toutes les cellules + bruit) ;
- N : nombre de RB de la bande de mesure (il normalise le rapport).

Le RSRQ dit donc **quelle part de ce que je reçois vient de ma cellule**. Plage : −19,5 à −3 dB. Au-dessus de −10 dB c'est bon ; en dessous de −15 dB la qualité est mauvaise.

Différence clé avec le RSRP : deux UE peuvent avoir le même RSRP de −95 dBm, l'un dans une zone calme (RSRQ −8 dB) et l'autre noyé dans l'interférence (RSRQ −17 dB). Le RSRQ voit la **charge et l'interférence**, le RSRP non.

> 🔧 **Simplification du simulateur :** le projet calcule RSRQ ≈ RSRP_service / Σ RSRP_toutes cellules (en mW), c'est-à-dire la part de la cellule dans la puissance reçue des 4 cellules, bornée à [−20, −3] dB. Ni la charge ni le facteur N ne sont modélisés exactement. Le RSRQ reste corrélé à la proximité de la frontière, ce qui est l'usage qu'en fait le modèle.

3.4 SINR — *Signal to Interference plus Noise Ratio*

$$SINR = \frac{S}{I + N}$$

C'est la grandeur physique qui détermine **combien de bits on peut transmettre** (théorème de Shannon : $C = B \log_2(1 + SINR)$). En LTE, le SINR n'est pas un rapport de mesure standardisé (il le devient en 5G NR

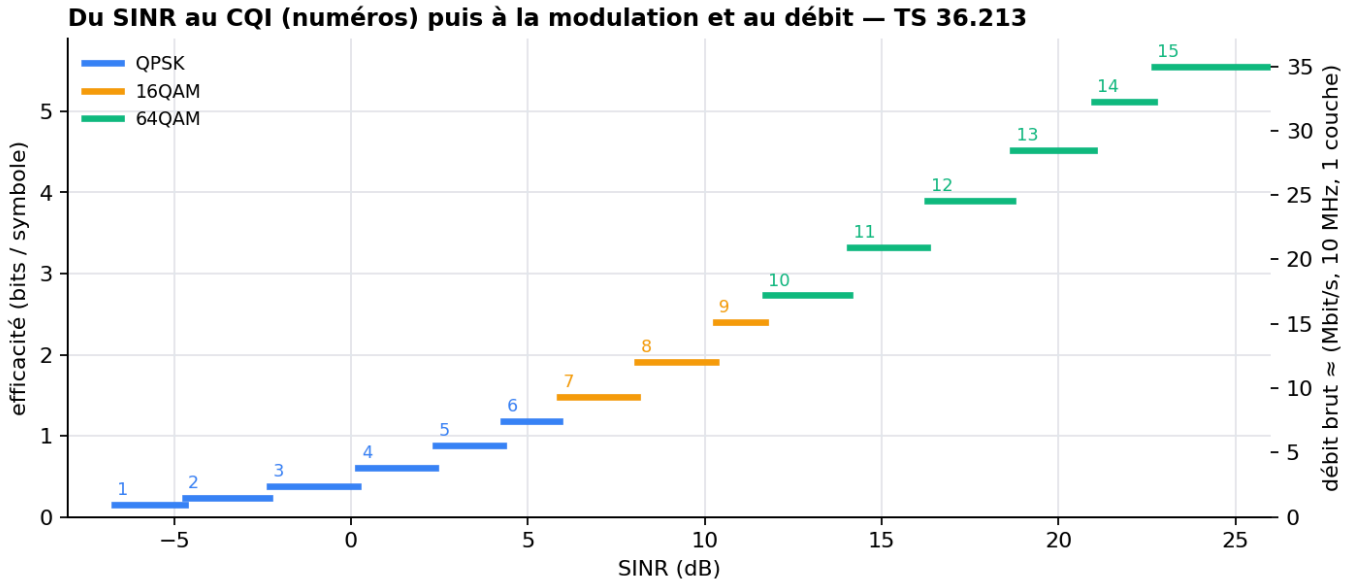
avec le SS-SINR), mais tous les terminaux l'estiment en interne pour calculer le CQI.

SINR	Qualité
> 20 dB	excellente
13 à 20 dB	bonne
0 à 13 dB	moyenne
< 0 dB	mauvaise (l'interférence domine)

Dans le simulateur, le SINR est calculé à partir des **RSRP filtrés L3** (comme un terminal réel qui moyenne ses estimations), avec une interférence pondérée par la charge de chaque voisine (chapitre 2). Pendant une **RLF** (*radio link failure*), il est forcé à −15 dB.

3.5 CQI — *Channel Quality Indicator*

Le CQI est un **entier de 1 à 15** que l'UE renvoie à l'eNodeB. Il signifie : « avec ce canal, voici le schéma de modulation et de codage le plus efficace que je peux décoder avec moins de 10 % d'erreurs de bloc ». L'ordonnanceur (couche MAC) s'en sert pour choisir la **modulation (QPSK, 16QAM, 64QAM)** et le **taux de codage**, donc le débit. C'est l'**adaptation de lien**.



Le CQI est une **version quantifiée du SINR** (table de TS 36.213). Il apporte peu d'information en plus du SINR, mais il est disponible côté réseau sans calcul. Le modèle l'utilise donc comme une variable de plus.

3.6 Pourquoi ces KPI, et pas d'autres ?

C'est une question d'entretien très probable. Quatre critères justifient le choix.

1. Ils sont disponibles là où la décision est prise, en temps réel. L'eNodeB reçoit les **rapports de mesure RRC** (*Measurement Reports*) qui contiennent le RSRP et le RSRQ de la cellule de service et des voisines. Il reçoit aussi le CQI en continu. Un modèle déployé dans l'eNodeB (ou un contrôleur SON/RIC) a donc accès à ces valeurs **sans rien changer au terminal ni à la norme**.

2. Ils sont standardisés. Leur définition est la même chez tous les constructeurs (TS 36.214, 36.133). Un modèle entraîné sur ces grandeurs est transposable d'un réseau à l'autre, ce qui n'est pas le cas d'un compteur propriétaire.

3. Ce sont des précurseurs physiques du handover. Le handover est déclenché par une comparaison de RSRP (événement A3). Avant ce déclenchement, le RSRP de service baisse, celui de la voisine monte, le SINR chute. Les KPI radio **précèdent causalement** l'événement à prédire.

4. Leur dynamique est informative. Ce n'est pas seulement la valeur, mais la **tendance** (le RSRP baisse-t-il de 2 dB/s ?) qui annonce un handover. D'où les variables de retard (*lags*), de moyennes glissantes et de dérivées (*deltas*) du chapitre 6.

Et les autres indicateurs ?

Indicateur	Pourquoi il n'est pas utilisé comme entrée
Débit (Mbit/s)	dépend du trafic de l'utilisateur et de l'ordonnanceur : un utilisateur qui ne télécharge rien a un débit nul même avec un excellent canal. Il **suit** la qualité radio avec retard.
Latence, gigue	indicateurs de bout en bout, influencés par tout le réseau (cœur, Internet), pas par la position radio.
BLER (taux d'erreurs de bloc)	volontairement maintenu autour de 10 % par l'adaptation de lien : il reste stable tant que le lien tient.
Position GPS (`pos_x` , `pos_y`)	**non disponible** pour le réseau en temps réel (confidentialité, batterie, précision en intérieur). Le simulateur la connaît, mais l'utiliser serait **tricher**. La v4 l'a retirée des entrées.
État LOS/NLOS (`los_flag`)	grandeur cachée : aucun équipement ne la mesure directement. Retirée des entrées pour la même raison.
Timing Advance	indique la distance à l'antenne de service, mais pas la position par rapport aux voisines. Possible extension.

> 🎯 **Phrase à retenir :** « J'utilise uniquement des grandeurs que l'eNodeB reçoit déjà dans les rapports de mesure standardisés. Le modèle est donc déployable sans modifier le terminal ni la norme. J'ai d'ailleurs retiré la position et l'état LOS, que le simulateur connaît mais qu'un vrai réseau n'observe pas. Mesuré sur la v3 du projet, ce retrait ne coûtait qu'environ 0,01 de F1. »

3.7 La vitesse et la charge

Deux variables de contexte complètent les KPI radio :

- `ue_speed` : la vitesse réelle du terminal (0 quand il est à l'arrêt). Un réseau peut l'estimer (effet Doppler, fréquence des handovers, *mobility state estimation* de la norme). Elle influence le TTT (chapitre 4) et la probabilité d'échec.
- `cell_load_pct` : la charge de la cellule de service (UE servis / capacité). Elle est connue de l'eNodeB et module l'interférence.
- `rsrp_diff` : RSRP service − RSRP meilleure voisine. C'est la grandeur la plus directement liée à A3, calculée à partir des deux RSRP.

3.8 Ce qu'il faut retenir

- **RSRP** = puissance d'un RE de référence : mesure de **couverture**, indépendante de la charge. C'est la grandeur d'A3.
- **RSRQ** = part de la cellule dans tout le reçu : mesure de **qualité**, sensible à l'interférence.
- **SINR** = rapport utile/nuisible qui détermine le débit ; il **s'effondre en bord de cellule**.
- **CQI** = SINR quantifié pour l'adaptation de lien.
- Ces KPI sont choisis parce qu'ils sont **disponibles, standardisés, précurseurs causaux** du handover, et informatifs par leur **dynamique**.

Chapitre 4 — Mobilité et handover en LTE

> **Objectif :** savoir dérouler la procédure de handover de bout en bout (mesure, déclenchement, décision, exécution), connaître les événements de mesure et les paramètres (offset, hystérésis, TTT), et comprendre les échecs que le projet cherche à anticiper.

4.1 Qu'est-ce qu'un handover ?

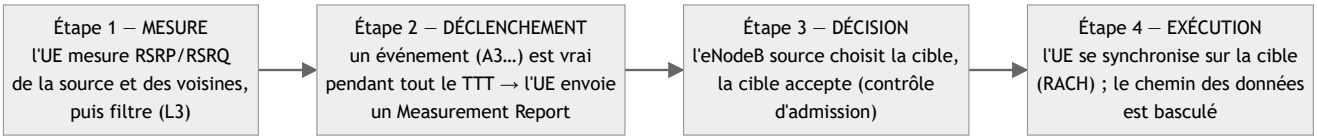
Un **handover** (HO, *transfert intercellulaire*) est le transfert d'une communication en cours d'une cellule (la **source**, ou *serving*) vers une autre (la **cible**, ou *target*), parce que la cible offre désormais une meilleure liaison radio.

En LTE, le handover est **« dur »** (*hard handover*, *break-before-make*) : l'UE coupe la liaison avec la source **avant** d'établir celle avec la cible. L'interruption dure quelques dizaines de millisecondes si tout se passe bien. S'il échoue, l'UE perd la liaison (**RLF**) et doit se reconnecter, ce qui coupe la communication pendant une à plusieurs secondes.

Les types de handover

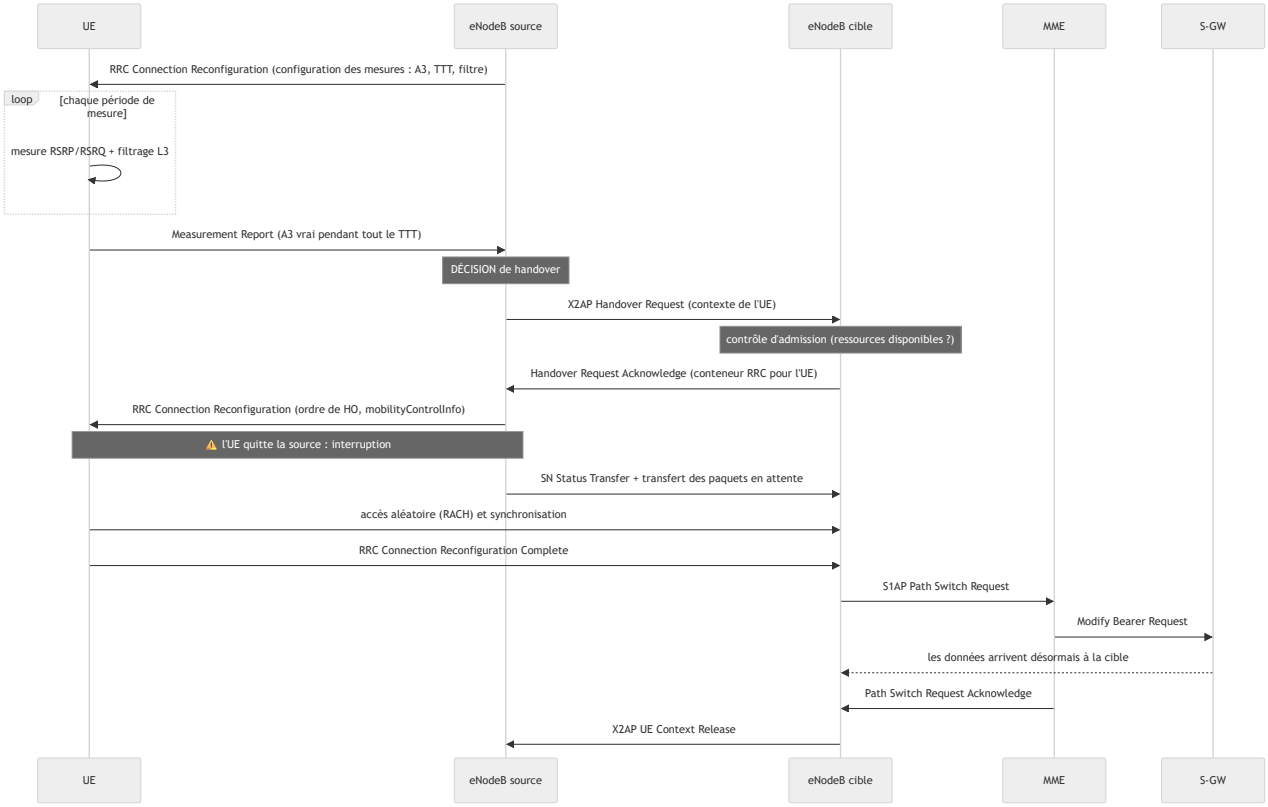
Type	Entre	Chemin de signalisation	Dans le projet
Intra-LTE, intra-fréquence, X2	deux eNodeB voisins, même fréquence	direct par **X2**	✅ c'est le cas modélisé
Intra-LTE, S1	deux eNodeB sans liaison X2	via le **MME** (S1)	—
Inter-fréquence	deux fréquences LTE	mesures en *gaps*	—
Inter-RAT	LTE ↔ 3G/2G/5G	via les cœurs de réseau	—

4.2 Les quatre étapes d'un handover



Le projet intervient entre l'étape 1 et l'étape 2 : il prédit, à partir des mesures, qu'un handover réussi aura lieu dans les 3 prochaines secondes, **avant** que l'événement ne se déclenche.

4.3 La procédure X2 en détail



Ce qui prend du temps et peut échouer :

- entre le *Measurement Report* et l'ordre de handover, il faut l'échange X2 et le contrôle d'admission (quelques dizaines de ms) ;
- l'**ordre de handover** est envoyé **par la source**, au moment où le lien avec elle se dégrade. Si le SINR est trop bas, l'UE ne le reçoit jamais : c'est le **handover trop tardif** ;
- l'accès à la cible (RACH) peut échouer si la cible est encore trop faible : c'est le **handover trop précoce**.

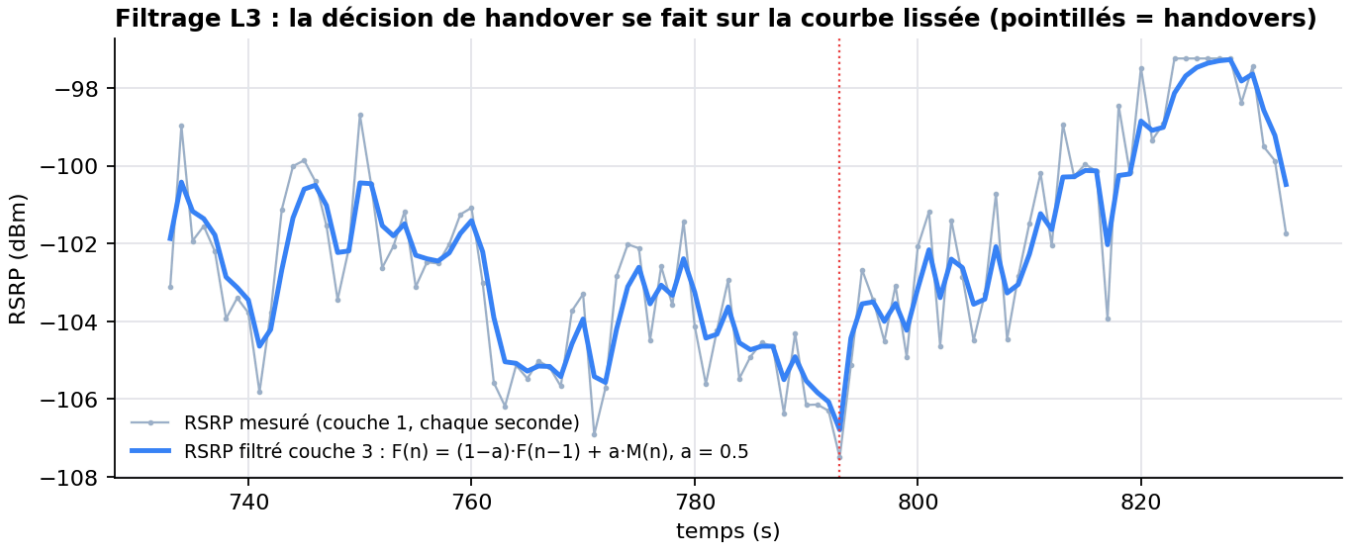
> 🎯 **D'où l'intérêt de la prédiction :** si l'eNodeB sait **3 secondes à l'avance** qu'un handover va avoir lieu, il peut **préparer la cible** (réserver des ressources, transférer le contexte), ou **avancer** le handover pendant que le SINR est encore bon. C'est exactement ce que teste l'expérience en boucle fermée du chapitre 9.

4.4 Le filtrage de couche 3

Les mesures brutes (couche 1) sont très bruitées. Avant de les comparer aux seuils, la couche RRC applique un **filtre exponentiel** (TS 36.331, §5.5.3.2) :

$$F_n = (1 - a)\, F_{n-1} + a\, M_n \qquad a = \frac{1}{2^{k/4}}$$

où M_n est la dernière mesure, F_n la valeur filtrée et k le *filterCoefficient* configuré par le réseau. **La valeur par défaut $k = 4$ donne $a = 0{,}5$**, celle du projet (`HOConfig.l3_alpha = 0.5`).



Un a petit (k grand) lisse beaucoup : moins de faux déclenchements, mais une réaction plus lente. C'est le premier compromis réactivité / stabilité de la mobilité.

4.5 Les événements de mesure

Le réseau configure **quand** l'UE doit envoyer un rapport, sous forme d'**événements** (TS 36.331, §5.5.4). « Serving » est la cellule de service ; « Neighbour » une voisine.

Événement	Condition d'entrée (simplifiée)	Usage typique
A1	Serving devient meilleure qu'un seuil	arrêter les mesures inter-fréquence
A2	Serving devient pire qu'un seuil	démarrer les mesures inter-fréquence
A3	**Neighbour devient meilleure que Serving + offset**	**handover intra-fréquence (le plus courant)**
A4	Neighbour devient meilleure qu'un seuil	handover vers une couche de capacité
A5	Serving pire que seuil 1 **et** Neighbour meilleure que seuil 2	handover de couverture (bord de cellule)
A6	Neighbour meilleure qu'une SCell + offset	agrégation de porteuses
B1 / B2	équivalents d'A4 / A5 vers une autre technologie	handover inter-RAT

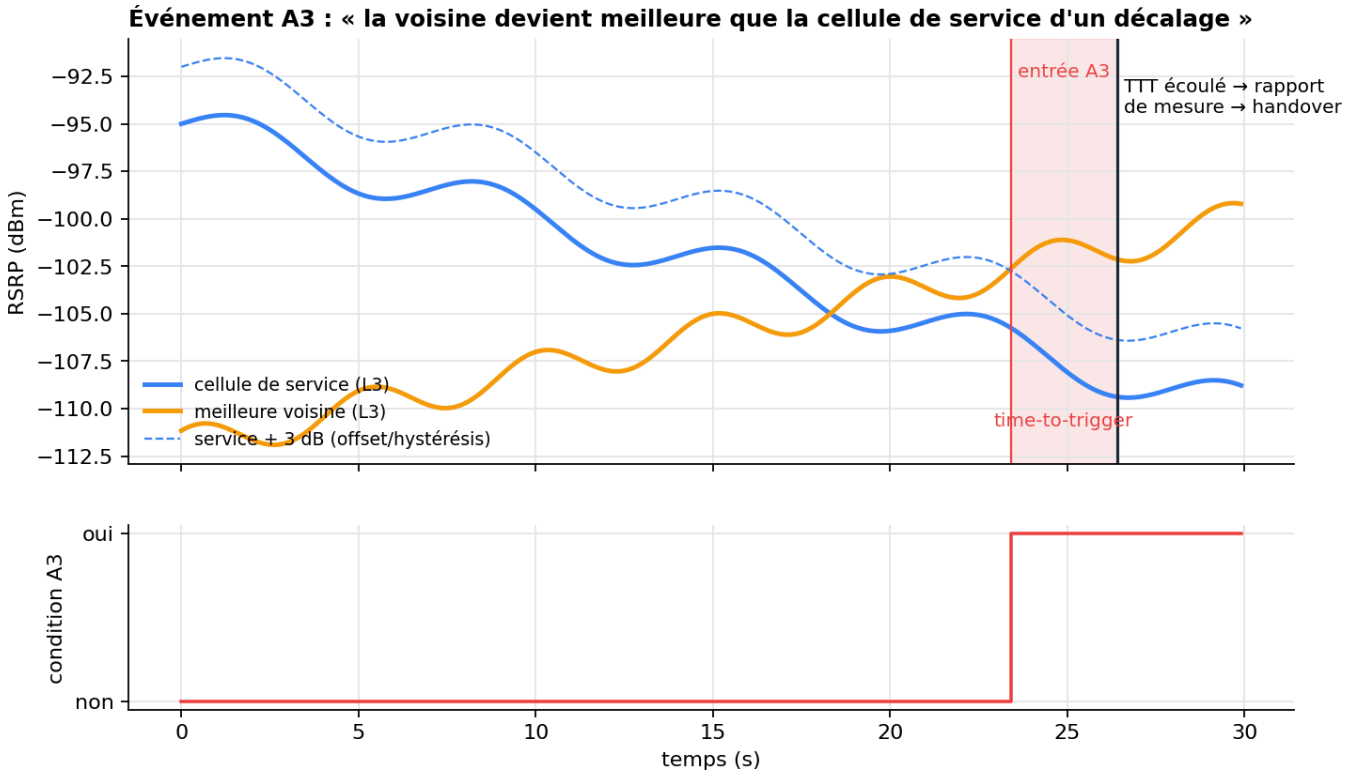
L'événement A3 en détail

La condition complète est :

$$M_n + O_{fn} + O_{cn} - Hys > M_p + O_{fp} + O_{cp} + Off$$

- M_n, M_p : mesures filtrées (RSRP) de la voisine et de la cellule de service ;
- O_f, O_c : décalages par fréquence et par cellule (*Cell Individual Offset*, CIO). Le CIO permet d'**avancer ou retarder** les handovers vers une voisine précise ;
- Hys : **hystérésis**, pour éviter les allers-retours ;
- Off : **offset** A3.

La condition doit rester vraie pendant tout le **Time-To-Trigger (TTT)** pour que le rapport soit envoyé. Valeurs normalisées du TTT : 0, 40, 64, 80, 100, 128, 160, 256, 320, 480, 512, 640, 1024, 1280, 2560, 5120 ms.



Dans le projet (`src/handover_logic.py`) : offset et hystérésis sont regroupés en une **marge de 3 dB** (`ho_margin_db`), et la comparaison se fait sur les **RSRP filtrés L3**. Le pas de temps est d'une seconde, donc le TTT est compté en secondes (1 à 3 s, plus grossier que la réalité).

TTT adapté à la vitesse

Un véhicule rapide traverse la zone de recouvrement en quelques secondes : un TTT long le ferait partir trop tard. La norme prévoit des **états de mobilité** (*normal / medium / high*), estimés d'après le nombre de handovers récents, qui **multiplient le TTT par un facteur de réduction** (*speed-dependent scaling*, sf-Medium, sf-High). Le projet reproduit cette idée avec une formule continue :

$$TTT = \max\left(1,\ 3 - \left\lfloor \frac{v}{7} \right\rfloor\right) \text{ secondes}$$

Soit 3 s pour un piéton, 2 s entre 7 et 14 m/s, et 1 s au-delà de 14 m/s.

Et A4 / A5 dans le projet ?

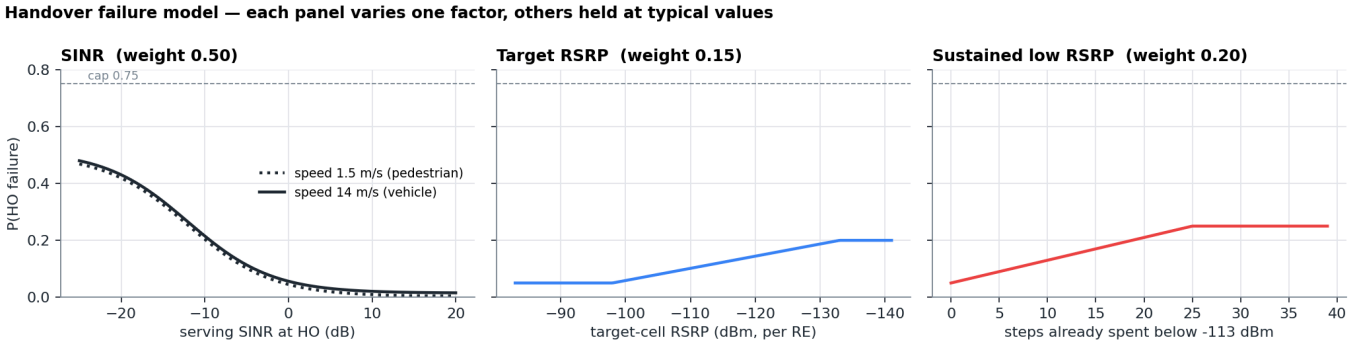
> ⚠️ **Nuance importante, à dire honnêtement si on vous le demande.** Dans le simulateur, **seul A3 déclenche** le handover. Quand le TTT expire, la logique **qualifie** l'événement selon la condition dominante : « A5 » si la source est sous −96 dBm et la voisine au-dessus de −88 dBm, « A4 » si la voisine dépasse −83 dBm, « A3 » sinon. La colonne `event_type` est donc une **étiquette descriptive** (handover de couverture, vers une cellule forte, ou relatif), pas un déclencheur indépendant. Dans un vrai réseau, A4 et A5 sont configurés comme des déclencheurs à part entière.

4.6 Quand le handover échoue

La RLF (*Radio Link Failure*)

L'UE surveille la qualité de son lien. Après **N310** indications consécutives de « hors synchronisation », il lance un temporisateur **T310**. Si le lien ne se rétablit pas avant son expiration, il déclare une **RLF**, puis tente un **RRC Connection Re-establishment** vers la meilleure cellule qu'il trouve. Pendant ce temps, la communication est coupée.

Dans le projet : un handover échoue avec une probabilité qui combine quatre facteurs (figure ci-dessous) : SINR au moment du handover (poids 50 %), vitesse (15 %), RSRP de la cible (15 %) et temps déjà passé à très faible RSRP (20 %). En cas d'échec, l'UE passe **exactement 3 s en RLF** (SINR forcé à −15 dB, aucun handover possible), puis **se rétablit sur la cellule la plus forte** (colonne `rrc_reestablish`). La v3 avait une RLF de 2 s par erreur et laissait l'UE sur la cellule défaillante ; la v4 corrige les deux.



Les quatre types de problèmes de mobilité

Problème	Ce qui se passe	Cause	Remède
Trop tardif (*too late*)	RLF dans la source **avant ou pendant** le handover	la condition est atteinte trop tard (TTT/offset trop grands, UE rapide)	avancer : réduire TTT/offset, augmenter le CIO de la cible
Trop précoce (*too early*)	RLF juste **après** le handover, rétablissement dans la source	la cible n'était pas encore assez forte	retarder : augmenter TTT/offset
Mauvaise cellule	RLF après handover, rétablissement dans une **troisième** cellule	mauvais choix de cible	ajuster les CIO
Ping-pong	handover A → B puis retour B → A en quelques secondes	fluctuations autour de la frontière	augmenter hystérésis/TTT

Ces problèmes **tirent dans des directions opposées** : réduire le TTT évite les handovers tardifs mais crée des ping-pongs, l'augmenter fait l'inverse. **C'est le compromis central de la mobilité.** L'expérience en boucle fermée du chapitre 9 le met en évidence de façon spectaculaire.

Dans le projet : un ping-pong est un handover réussi qui annule un handover réussi effectué dans les 10 dernières secondes. Quand il est détecté, la paire inverse reçoit **3 dB de marge supplémentaire pendant 20 s** (hystérésis adaptative). Depuis la v4, seuls les handovers **réellement exécutés** entrent dans cet historique (les tentatives échouées n'y figurent plus).

4.7 SON et MRO : où se place le projet

Les réseaux modernes intègrent des fonctions **SON** (*Self-Organizing Networks*, 3GPP TS 32.500). L'une d'elles, le **MRO** (*Mobility Robustness Optimisation*), compte les handovers trop tardifs, trop précoces et les ping-pongs, puis ajuste automatiquement les CIO, l'hystérésis et le TTT de chaque paire de cellules.

Le MRO classique est **réactif** : il corrige après avoir observé des échecs, sur des périodes de plusieurs minutes ou heures. **Ce projet explore une approche prédictive** : anticiper, UE par UE et seconde par seconde, qu'un handover est imminent. C'est l'esprit des architectures **O-RAN**, où des applications d'IA (*xApps*) tournent dans un contrôleur radio (*RIC*) proche des eNodeB/gNodeB.

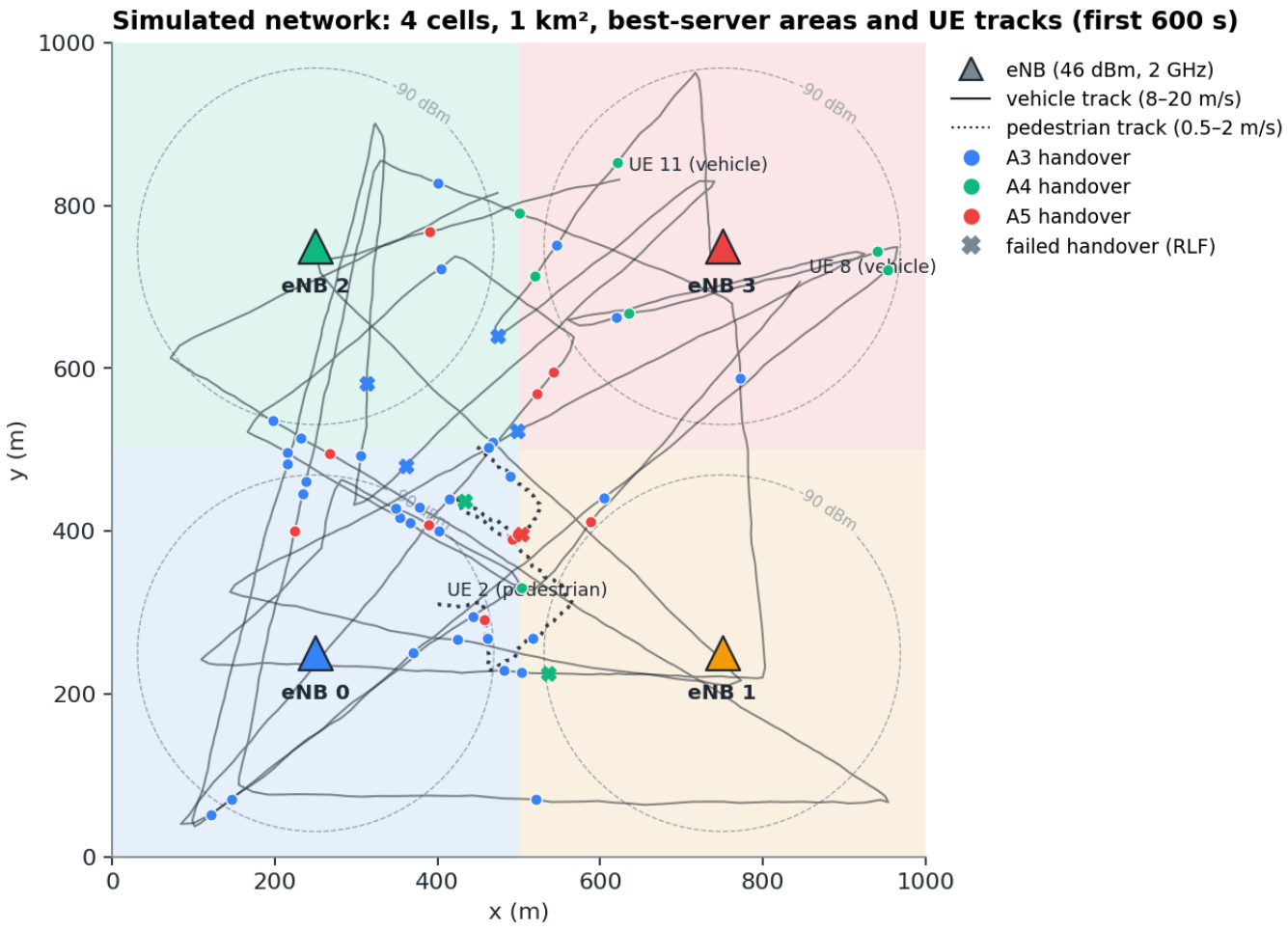
4.8 Ce qu'il faut retenir

- Le handover LTE est **dur** ; en intra-fréquence il passe par **X2** et c'est l'**eNodeB source qui décide**, sur **Measurement Report**.
- Chaîne : **mesure → filtrage L3 → événement (A3) vrai pendant le TTT → rapport → décision → exécution**.
- **A3** : voisine > service + offset/hystérésis, pendant le **TTT**. TTT réduit pour les UE rapides.
- Échecs : **trop tardif, trop précoce, mauvaise cellule, ping-pong** ; une **RLF** coupe la communication.
- Le compromis **réactivité vs stabilité** (TTT court vs long) est au cœur du MRO… et des résultats du projet.

Chapitre 5 — Le simulateur du projet

> **Objectif :** relier chaque concept des chapitres 1 à 4 au code, savoir ce qui est fidèle à la réalité et ce qui est simplifié, et pouvoir défendre les corrections de la version 4.

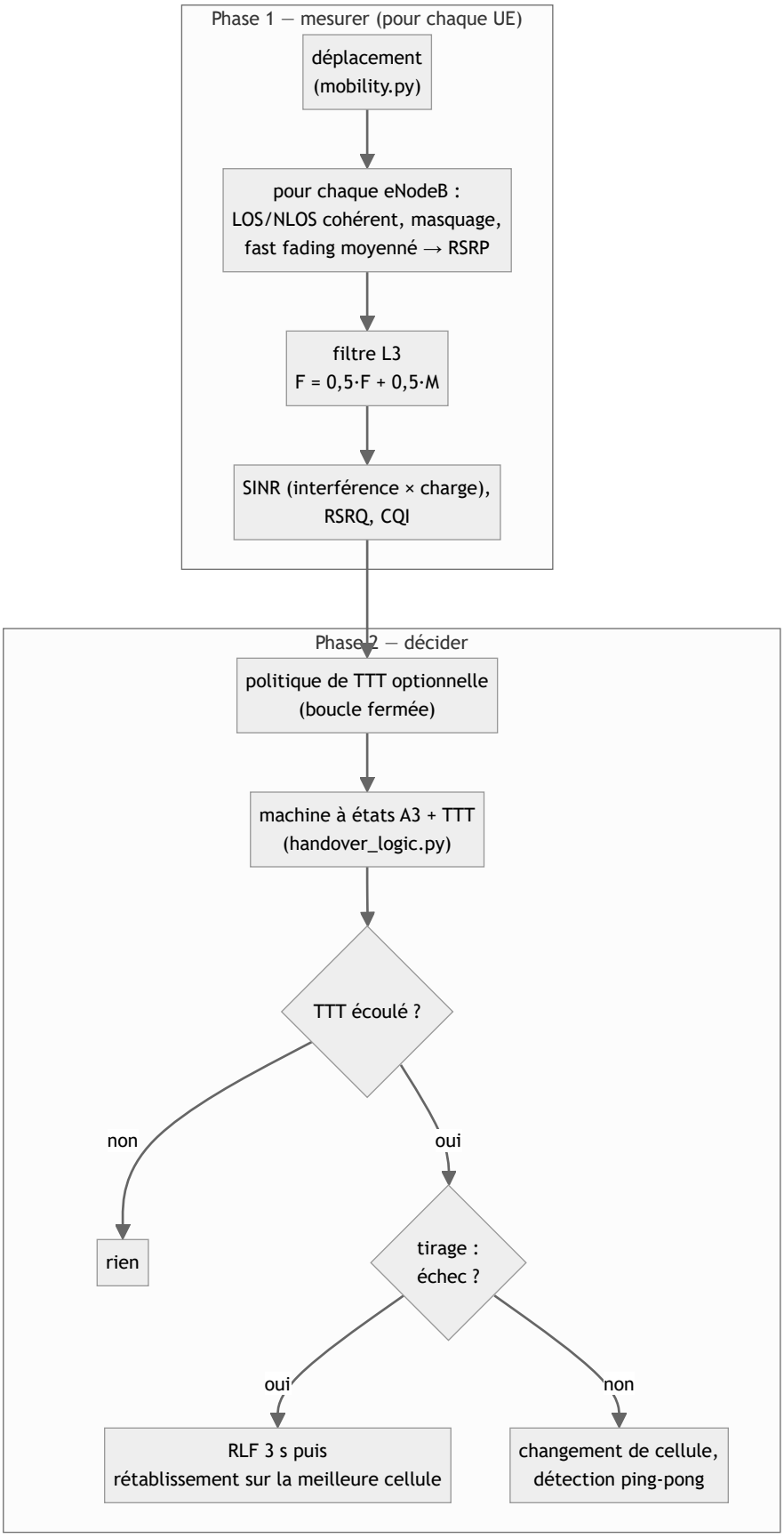
5.1 Ce qui est simulé



Élément	Valeur	Commentaire
Zone	1 km × 1 km	ville dense (*Urban Macro*)
eNodeB	4, aux positions (250, 250), (750, 250), (250, 750), (750, 750) m	distance inter-sites 500 m, typique d'une macro-cellule urbaine
Émission	46 dBm (40 W) à 2 GHz, 10 MHz (50 RB)	soit **18,2 dBm par RE** pour le RSRP
UE	8 piétons (0,5–2 m/s) + 7 véhicules (8–20 m/s)	nouvelle vitesse tirée à chaque trajet (*waypoint*)
Mobilité	*Random Waypoint* avec persistance de direction	piétons : trajets courts (≤ 200 m), pauses ≤ 10 s ; véhicules : toute la grille
Pas de temps	1 s	une mesure, une décision de handover par seconde
Durée	3 600 s	54 000 lignes

5.2 Une seconde de simulation, étape par étape

Le cœur est la classe `Simulator` (`src/simulator.py`). Chaque seconde se déroule en deux phases :



Deux générateurs aléatoires indépendants sont utilisés : l'un pour la mobilité et le canal radio, l'autre pour les tirages d'échec de handover. Ainsi, deux simulations avec la même graine voient **exactement les mêmes trajectoires et le même canal**, même si une politique change les décisions de handover. C'est ce qui rend la comparaison du chapitre 9 rigoureuse (un test le vérifie).

5.3 Du concept au code

Concept (chapitre)	Fonction / paramètre	Valeur
Path loss UMa LOS / NLOS (2)	`path_loss_los` , `path_loss_nlos`	TR 36.873
Probabilité LOS (2)	`p_los`	$\min(18/d, 1)(1 - e^{-d/63}) + e^{-d/63}$
LOS cohérent dans l'espace (2)	`update_los_latent` , `los_from_latent`	décorrélation 50 m (TR 38.901)
Masquage (2)	`update_shadow`	σ 4 / 6 dB, décorrélation 100 m
Fast fading moyenné (2)	`update_fast_fading`	1 échantillon par λ/2 parcouru
RSRP par RE (3)	`compute_rsrp` , `RadioConfig.rs_power_dbm`	46 − 27,8 = 18,2 dBm
Plage de rapport (3)	`report_rsrp`	[−140, −44] dBm
SINR (2, 3)	`compute_sinr`	interférence × (0,1 + 0,9·charge), bruit −95,5 dBm
RSRQ, CQI (3)	`compute_rsrq` , `sinr_to_cqi`	table TS 36.213
Filtre L3 (4)	`l3_update` , `HOConfig.l3_alpha`	a = 0,5 (k = 4)
A3 + marge (4)	`evaluate_handover` , `ho_margin_db`	3 dB
TTT selon la vitesse (4)	`velocity_ttt`	max(1, 3 − ⌊v/7⌋) s
Échec du handover (4)	`ho_failure_prob`	SINR 50 %, vitesse 15 %, cible 15 %, durée en faible RSRP 20 % ; plafond 75 %
RLF et rétablissement (4)	`Simulator._decide`	3 s exactement, puis meilleure cellule
Ping-pong (4)	`is_ping_pong` , `pp_extra_margin_db`	retour en < 10 s ; +3 dB pendant 20 s
Label (6)	`add_handover_soon_label`	handover réussi dans les 3 s

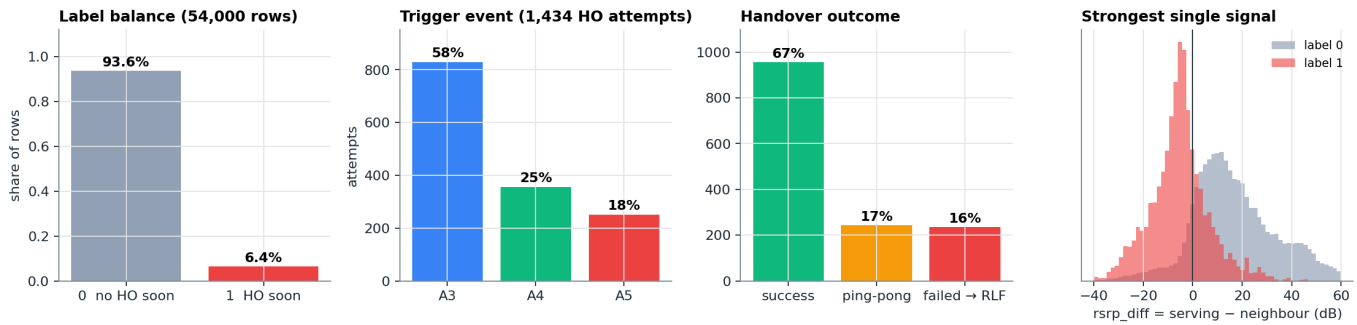
5.4 Les corrections de la version 4

La revue du projet a relevé des écarts à la réalité. Les savoir expliquer montre qu'on maîtrise à la fois la télécom et l'esprit critique sur ses propres données.

#	Problème en v3	Pourquoi c'est faux	Correction v4	Effet mesuré
1	RSRP calculé avec les 46 dBm totaux	le RSRP est la puissance **d'un RE**	puissance par RE (−27,8 dB), seuils décalés d'autant, plage −44 dBm	0 valeur au-dessus de −44 dBm (20 % avant) ; SINR inchangé
2	LOS retiré toutes les 5 s, indépendamment du mouvement	un piéton immobile ne passe pas de LOS à NLOS	processus latent corrélé à la distance (TR 38.901)	handovers piétons : **114** → **~38 par heure** ; ping-pong piétons 27 % → ~10 %
3	fast fading = un échantillon Rayleigh par seconde	un terminal **moyenne** ses mesures	moyenne de puissance sur 1 + Δd/(λ/2) échantillons	variations rapides ±5 dB → ~1 dB (piéton)
4	vitesse constante par UE, non nulle à l'arrêt	la vitesse servait d'identifiant d'UE	nouvelle vitesse par trajet, 0 à l'arrêt	la variable a un sens physique
5	RLF de 2 s au lieu de 3, UE laissé sur la cellule défaillante	erreur d'ordre des opérations	décrément après la décision ; rétablissement RRC	conforme à la configuration
6	tentatives échouées comptées dans l'historique ping-pong	un échec n'est pas un handover	seuls les handovers exécutés y entrent	détection correcte
7	bruit à −107 dBm	kTB sur 9 MHz = −104,5 dBm, + facteur de bruit	−95,5 dBm	négligeable (réseau limité par l'interférence)

> 🎯 **Comment le présenter :** « En relisant mon simulateur avec un œil de radio, j'ai trouvé qu'il calculait le RSRP avec la puissance totale au lieu de la puissance par RE, et que l'état LOS changeait même pour un terminal immobile, ce qui créait des handovers artificiels. J'ai corrigé, mesuré l'effet, et j'ai tout ré-entraîné. »

5.5 Le dataset obtenu



Statistique (seed 42, 1 h, 15 UE)	Valeur
Lignes	54 000
Secondes positives (`handover_soon = 1`)	**6,4 %**
Tentatives de handover	1 434 (≈ 96 par UE et par heure)
— réussies	1 199 (83,6 %)
— échouées → RLF	235 (16,4 %)
Ping-pong (parmi les réussies)	244 (20,4 %)
Étiquette d'événement	A3 57,7 % · A4 24,8 % · A5 17,5 %
Piétons / véhicules	38 / 161 tentatives par UE et par heure
Rétablissements RRC après RLF	209
Liens de service en LOS	24,4 % du temps
RSRP de service	de −131,5 à −44 dBm (0,8 % des valeurs au plafond de −44)
SINR médian	16,5 dB

Lecture. Les véhicules changent de cellule environ 4 fois plus souvent que les piétons, ce qui est logique : ils traversent les frontières plus vite. Le taux d'échec (~16 %) et le taux de ping-pong (~20 %) sont élevés pour un réseau réel optimisé. Ils reflètent un réseau **non optimisé** (marge et TTT fixes, aucun MRO), ce qui laisse justement de la place à une amélioration par la prédiction (chapitre 9).

5.6 Les limites à connaître (et à assumer)

Un simulateur reste un modèle. Il faut savoir dire ce qu'il ne fait pas :

- **Données synthétiques.** La vraie validation serait des traces réelles (drive tests, données OSS d'un opérateur).
- **Pas de temps d'une seconde.** Les vrais terminaux mesurent toutes les ~200 ms et le TTT descend à 40 ms ; le simulateur est plus grossier.
- **Transitions LOS franches.** Même avec la cohérence spatiale, passer de LOS à NLOS est un saut d'environ 20 dB. C'est la principale source restante de handovers piétons (~38 par heure, plus que dans un vrai réseau). Un essai de transition « douce » (*soft LOS*) a augmenté ce nombre au lieu de le réduire ; il n'a pas été retenu.
- **Réseau minuscule.** 4 cellules sans *wrap-around* : les UE près des bords voient moins d'interférence qu'au centre d'un vrai réseau.
- **Simplifications** : pas d'antennes sectorisées ni de diagramme d'antenne, RSRQ approché, une seule fréquence, A4/A5 utilisés comme étiquettes et non comme déclencheurs, modèle d'échec heuristique (calibré pour ~15–20 % d'échecs, valeur pessimiste par rapport à un réseau optimisé).

Chapitre 6 — Le problème de ML : label, données, variables

> **Objectif :** savoir formuler précisément ce que le modèle prédit et pourquoi, expliquer chaque variable d'entrée, et démontrer que le pipeline **ne triche pas** (pas de fuite de données).

6.1 La question posée au modèle

À chaque seconde t, pour chaque UE, le modèle répond à :

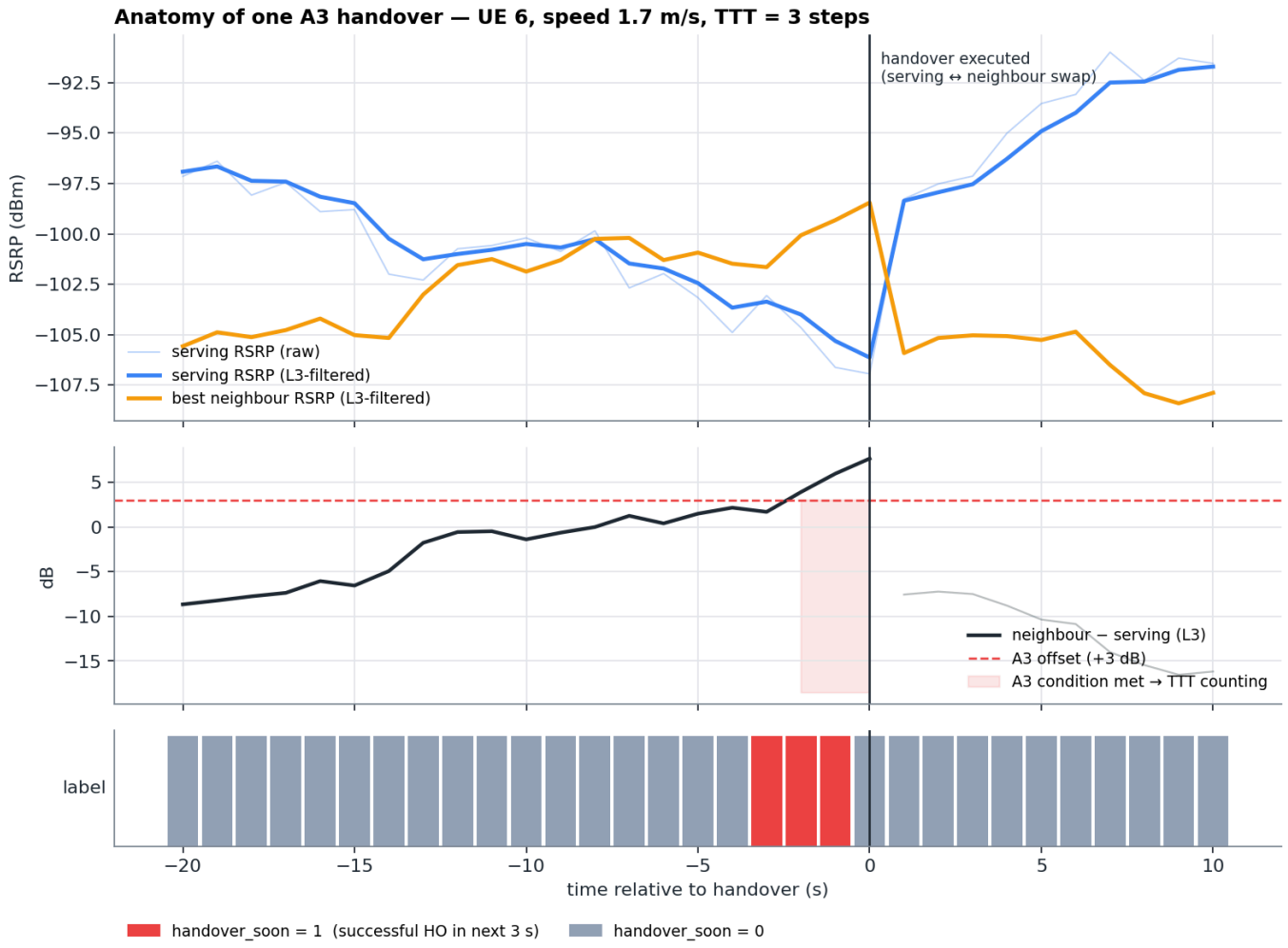
> **« Un handover *réussi* va-t-il avoir lieu pour cet UE dans les 3 prochaines secondes ? »**

C'est une **classification binaire** : la sortie est une probabilité $p \in [0, 1]$, transformée en décision (alarme / pas d'alarme) par un **seuil**.

Le label s'appelle `handover_soon` :

$$\text{handover_soon}(t) = 1 \iff \exists\, k \in \{1, 2, 3\} : \text{un handover réussi a lieu à } t + k$$

Il est calculé dans `src/simulator.py` (`add_handover_soon_label`).



La figure montre un vrai handover du dataset : le label (bandeau du bas) passe à 1 **trois secondes avant** le handover, souvent avant même que la condition A3 soit remplie. Le modèle doit donc reconnaître une **trajectoire** qui mène au handover, pas seulement l'état « A3 vrai ».

Pourquoi 3 secondes ?

Horizon	Avantage	Inconvénient
1 s	facile à prédire	trop tard pour préparer quoi que ce soit
3 s	**laisse le temps de préparer la cible (échanges X2 ~ dizaines de ms) ou d'avancer le handover**	**le label reste lié à des tendances radio observables**
10 s	très utile opérationnellement	la mobilité est imprévisible à cette échelle : bruit énorme dans le label

Pourquoi seulement les handovers *réussis* ?

Les tentatives qui échouent mènent à une RLF, qui est un autre phénomène (le lien casse). Les mélanger au label brouillerait la cible. Le modèle apprend à anticiper les **handovers qui ont vraiment lieu**.

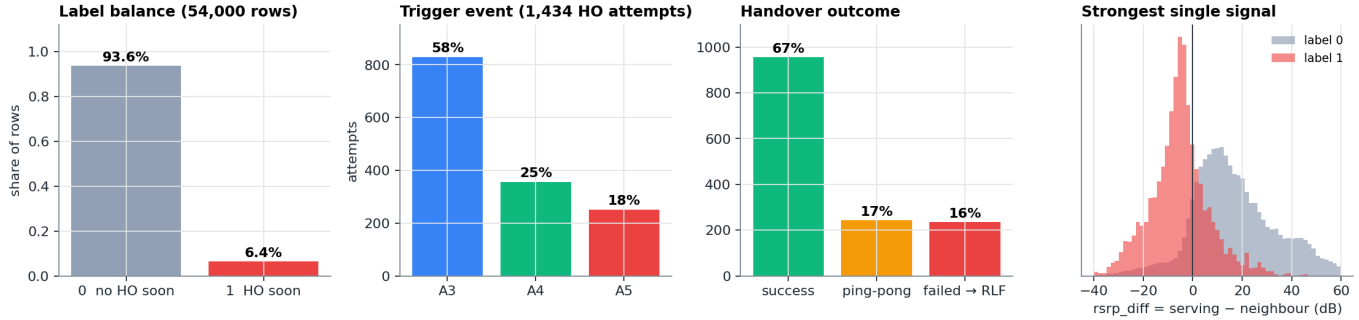
Un problème déséquilibré

Seuls **~6 % des secondes** sont positives : un UE passe la plupart de son temps loin d'un handover. Conséquences, détaillées aux chapitres 7 et 8 :

- l'**accuracy** est trompeuse (un modèle qui dit toujours « non » a 94 % d'accuracy) ;
- il faut **rééquilibrer l'apprentissage** (pondération des classes) ;
- il faut des métriques adaptées (**F1, PR-AUC**) et **choisir le seuil**.

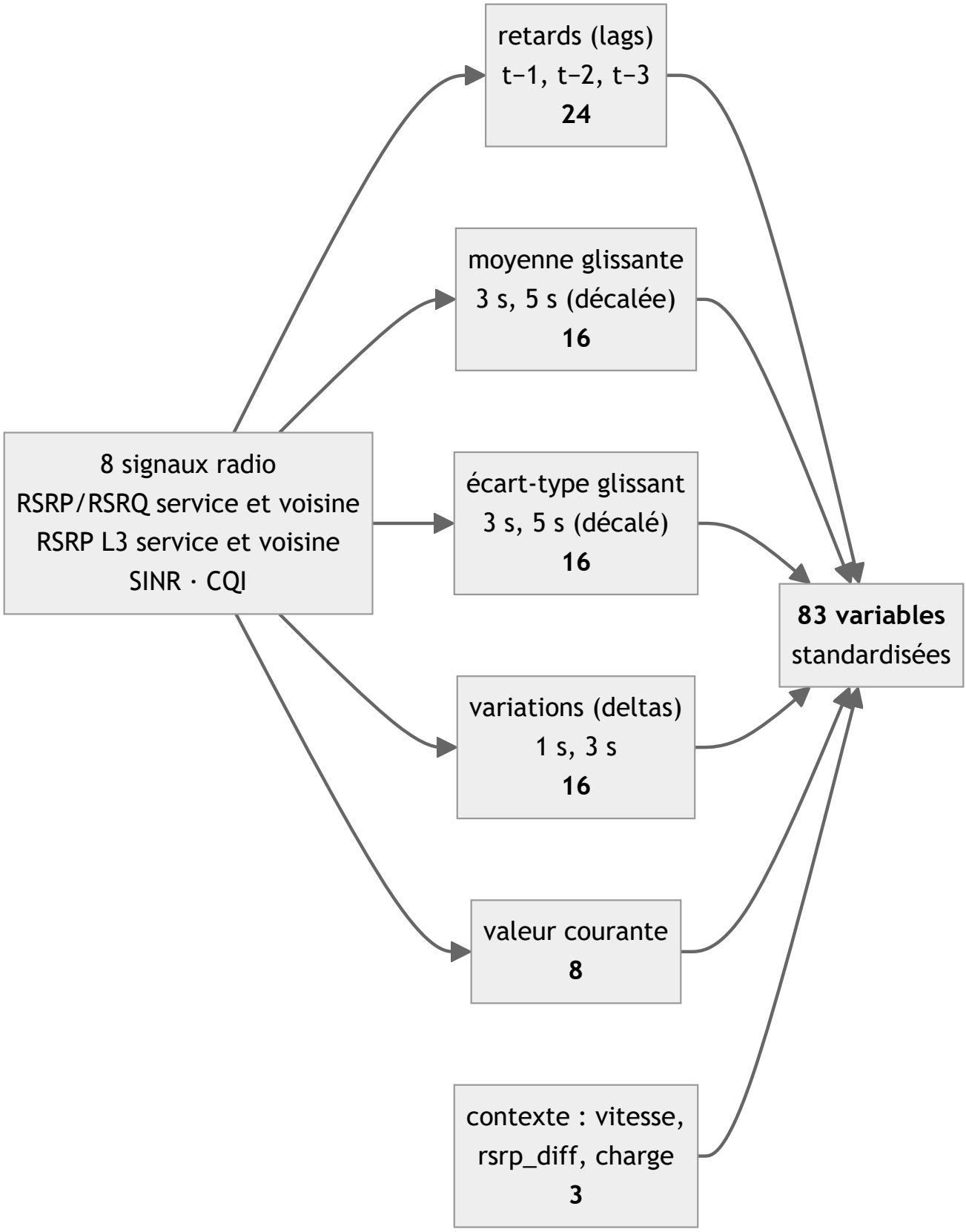
6.2 Les données

Élément	Valeur
Réseau	4 eNodeB sur une grille de 1 km², distance inter-sites 500 m
UE	15 : 8 piétons (0,5–2 m/s) et 7 véhicules (8–20 m/s)
Durée	3 600 s (1 h), pas de 1 s
Lignes	54 000 (15 UE × 3 600 s)
Colonnes brutes	27 (mesures, contexte, événements, label)
Graine aléatoire	42 (dataset reproductible à l'identique)



> 💡 **Pourquoi simuler ?** Il n'existe pas de dataset public de mesures LTE **étiquetées** par les handovers, au pas de la seconde, avec les voisines. Les traces réelles des opérateurs sont confidentielles. Le simulateur permet aussi des **expériences impossibles sur un vrai réseau** : rejouer exactement la même heure avec une autre politique de handover (chapitre 9).

6.3 Les variables d'entrée (83 features)



Pourquoi ces transformations ?

Un modèle « tabulaire » (régression logistique, forêt, XGBoost) ne voit qu'**une ligne à la fois** : il ne sait pas ce qui s'est passé avant. On lui **donne la mémoire** sous forme de colonnes.

Famille	Exemple	Ce que ça capture
Lag (retard)	`rsrp_serving_lag3` = RSRP il y a 3 s	l'état passé
Delta (variation)	`rsrp_serving_delta3` = RSRP(t) − RSRP(t−3)	la **tendance** : « le signal chute de 6 dB en 3 s »
Moyenne glissante	`sinr_roll5_mean`	le niveau de fond, débarrassé du bruit
Écart-type glissant	`rsrq_serving_roll5_std`	l'**instabilité** : un lien qui fluctue est souvent en bord de cellule

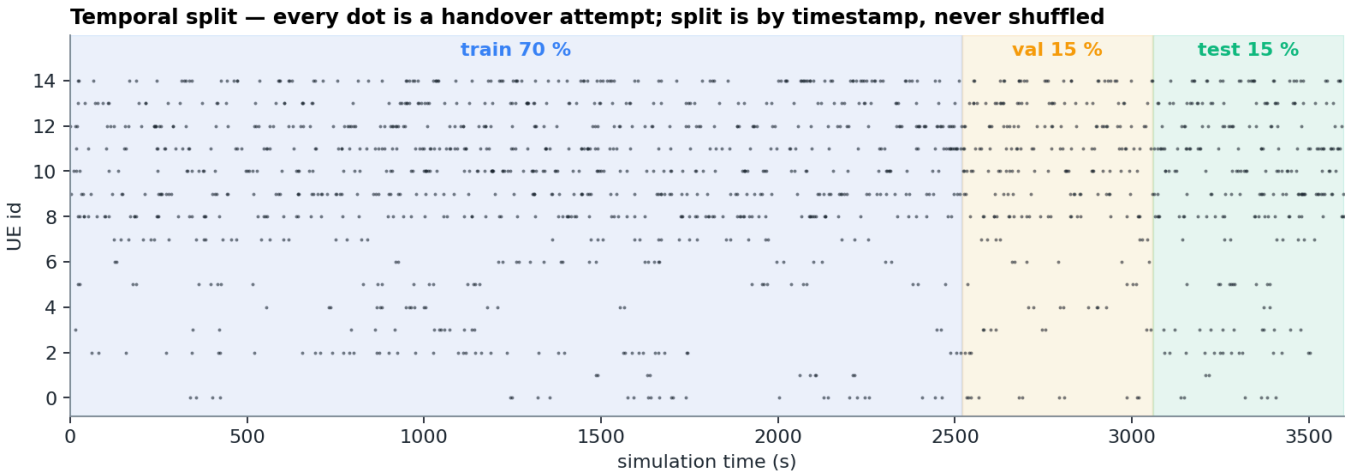
> 🎯 La tendance **complète** le niveau : un RSRP de −100 dBm stable n'annonce rien, alors qu'un RSRP qui passe de −90 à −100 dBm en 3 s pendant que la voisine monte annonce un handover. L'analyse SHAP (chapitre 8) montre que l'état courant (écart service/voisine, SINR, RSRQ) et la vitesse portent l'essentiel de la décision, et que les variables temporelles en apportent environ un tiers.

6.4 Pas de fuite de données : comment on le garantit

Une **fuite de données** (*data leakage*), c'est quand le modèle voit pendant l'entraînement une information qu'il n'aura pas en production. Ses scores sont alors faussement bons. Le projet en évite cinq formes.

1. Pas d'information du futur dans les variables. Les lags et les deltas ne regardent que le passé. Les fenêtres glissantes sont **décalées d'un pas** (`shift(1)`) : la moyenne sur 5 s à l'instant t utilise $t-5 \dots t-1$. Un test automatique (`tests/test_features.py`) réécrit le futur d'un UE et vérifie que les variables du passé ne bougent pas.

2. Un découpage temporel, jamais aléatoire. Les 70 % premières secondes servent à l'entraînement, les 15 % suivantes à la validation, les 15 % dernières au test. Un découpage aléatoire mettrait la seconde t en entraînement et la seconde $t+1$ (presque identique) en test, ce qui gonfle les scores.



3. Normalisation apprise sur l'entraînement seulement. Le `StandardScaler` (centrer-réduire chaque variable) calcule moyennes et écarts-types **sur le train uniquement**, puis les applique à la validation et au test.

4. Pas de variables qu'un réseau n'observe pas. La position (`pos_x` , `pos_y`) et l'état LOS (`los_flag`) existent dans le dataset (pour l'analyse et les figures) mais **ne sont pas des entrées du modèle** depuis la v4.

5. Sélection du modèle sans regarder le test. Seuils et choix du champion se font sur la **validation** ; le test n'est lu qu'à la fin (chapitre 8).

6.5 Du hors-ligne au temps réel

En production, les mesures arrivent seconde par seconde. Deux implémentations calculent **exactement les mêmes variables** :

- `build_features()` (hors ligne) : pour l'entraînement, `predict.py` et le dashboard. Elle accepte des lignes dans n'importe quel ordre, les regroupe par UE, les trie par temps, et **conserve l'index d'origine** pour rattacher chaque score à sa ligne ;
- `OnlineFeatureBuffer` (en ligne) : garde les 5 dernières mesures de chaque UE et calcule les variables de la nouvelle seconde. C'est lui qui tourne dans la boucle fermée (chapitre 9).

Un test vérifie que les deux donnent les mêmes valeurs (écart < 10^{-8}). Les **3 premières secondes** de chaque UE n'ont pas d'historique : elles sont marquées *warm-up* et ne sont pas notées, plutôt que d'être remplies avec des zéros.

> 🦎 **Anecdote utile en entretien :** la v3 avait deux bugs à cet endroit. `predict.py` ne calculait pas les variables temporelles et rejetait donc **100 % des lignes** d'un fichier de mesures brutes. Le dashboard recalculait les variables en réordonnant les lignes par UE, puis rattachait les scores **par position** : pour un fichier trié par temps, chaque risque était affiché sur la mauvaise ligne. La correction : un seul chemin d'inférence (`src/serving.py`), rattachement **par index**, et un test qui mélange l'ordre des lignes et vérifie que les scores ne changent pas.

6.6 Ce qu'il faut retenir

- Cible : **handover réussi dans les 3 s**, classification binaire **déséquilibrée** (~6 %).
- **83 variables** : 8 signaux × (valeur, 3 lags, 2 moyennes, 2 écarts-types, 2 deltas) + vitesse, `rsrp_diff` , charge.
- Les variables temporelles donnent la **mémoire** et la **tendance** aux modèles tabulaires.
- **Anti-fuite** : fenêtres décalées, split temporel, scaler sur le train, pas de position ni de LOS, sélection sur la validation.

Chapitre 7 — Les modèles

> **Objectif :** comprendre comment fonctionne chacun des sept modèles comparés, quels hyperparamètres ont été choisis et pourquoi, et comment le déséquilibre des classes est traité.

7.1 Vue d'ensemble

Famille	Modèle	Idée en une phrase	Voit l'historique comment ?
Baseline	**Règle A3**	probabilité = sigmoïde de l'écart L3 voisine − service	pas du tout (valeur instantanée)
Linéaire	**Régression logistique**	somme pondérée des variables → sigmoïde	via les lags/deltas
Arbres (bagging)	**Random Forest**	vote de 300 arbres entraînés sur des échantillons différents	via les lags/deltas
Arbres (boosting)	**XGBoost**	arbres ajoutés un à un pour corriger les erreurs des précédents	via les lags/deltas
Réseaux récurrents	**LSTM**, **GRU**	lisent une fenêtre de 10 s pas à pas, avec une mémoire interne	directement (séquence)
Ensemble	**Stacking**	une régression logistique combine XGBoost, RF et LSTM	via les modèles de base

> 💡 **Pourquoi plusieurs familles ?** Pour répondre à la question « quel gain apporte la complexité ? ». La règle mesure ce que donne la logique A3 seule ; la régression logistique, un modèle linéaire ; les arbres, les interactions non linéaires ; les LSTM, l'apprentissage direct de la dynamique. Si un modèle simple suffisait, il faudrait le préférer.

7.2 La baseline : une règle sans apprentissage

$$p = \frac{1}{1 + e^{-(\Delta_{L3} - 3)/2}} \qquad \Delta_{L3} = RSRP^{L3}_{voisine} - RSRP^{L3}_{service}$$

La probabilité vaut 0,5 quand la voisine dépasse la cellule de service de 3 dB (la marge A3), et monte quand l'écart grandit. C'est **ce que ferait un ingénieur sans ML**. Tout modèle appris doit faire mieux, sinon il ne sert à rien (`RuleA3Gap` dans `src/models.py`).

7.3 La régression logistique

Le modèle calcule un score linéaire, puis le passe dans une sigmoïde :

$$p = \sigma(w^\top x + b) = \frac{1}{1 + e^{-(w^\top x + b)}}$$

Les poids w minimisent la **perte logistique** (*log-loss*, entropie croisée), avec une **régularisation** L2 qui empêche les poids d'exploser ($C = 0{,}5$: plus C est petit, plus la régularisation est forte).

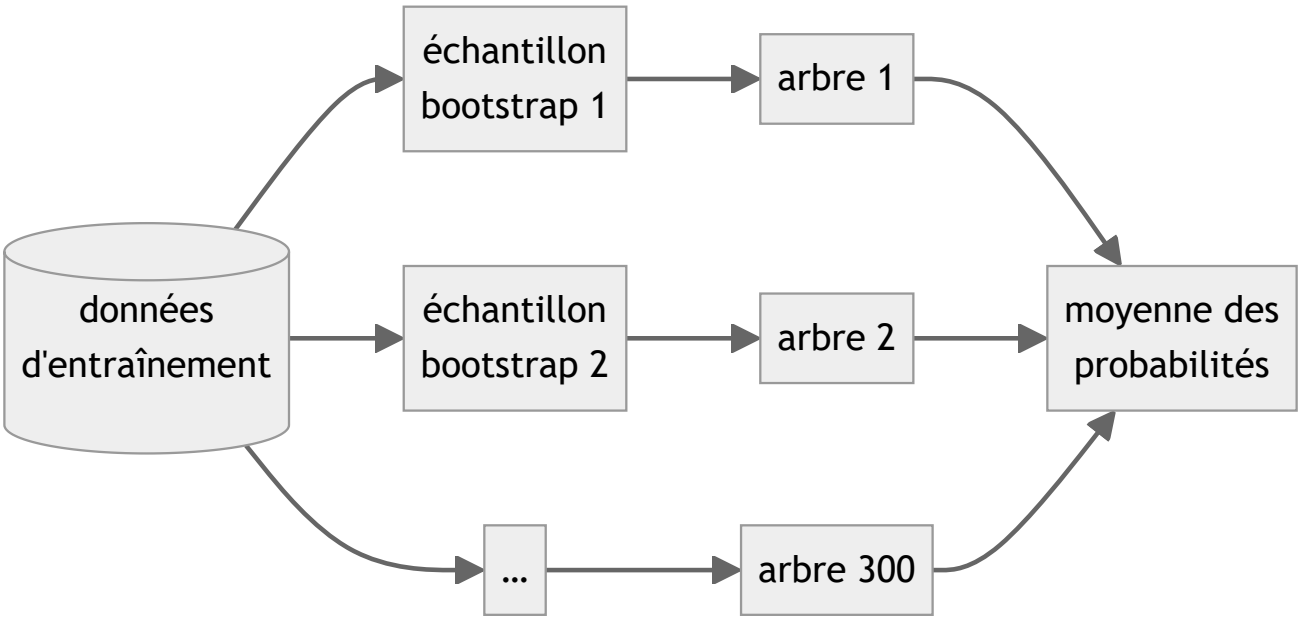
- ✅ rapide, interprétable (un poids par variable), bien calibré ;
- ❌ ne capte que des relations **linéaires** : il ne peut pas exprimer « si le RSRP baisse **et** que la voisine monte **et** que l'UE est rapide ».

7.4 Les arbres de décision, la forêt aléatoire et XGBoost

Un arbre de décision

Un arbre pose une suite de questions binaires (« `rsrp_diff` < −2 dB ? », « `sinr_delta3` < −4 dB ? ») et prédit en fonction de la feuille atteinte. Chaque question est choisie pour séparer au mieux les classes (critère de Gini ou d'entropie). Un arbre seul **sur-apprend** facilement : il finit par mémoriser le bruit.

Random Forest (bagging)



On entraîne **300 arbres**, chacun sur un tirage aléatoire des données (*bootstrap*) et en ne regardant qu'un sous-ensemble aléatoire de variables à chaque division. Leurs erreurs sont en partie indépendantes, donc **la moyenne réduit la variance**.

Hyperparamètres : `n_estimators=300`, `max_depth=12` (limite la complexité), `min_samples_leaf=5` (pas de feuille sur moins de 5 exemples).

XGBoost (boosting de gradient)



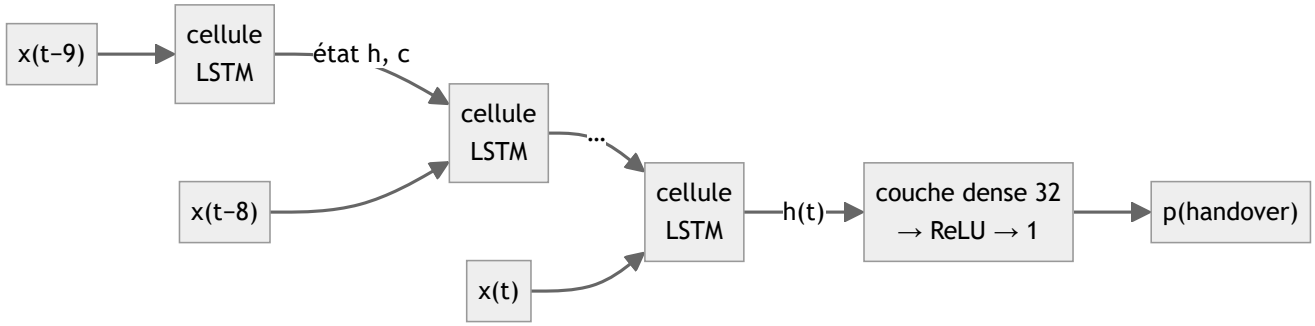
Ici les arbres sont **séquentiels** : chacun est ajusté sur le **gradient de la perte** des précédents, c'est-à-dire sur ce qu'ils ont encore mal prédit. Le *learning rate* (0,05) rend chaque correction petite, ce qui demande plus d'arbres mais généralise mieux.

Hyperparamètres : profondeur 6, `subsample=0.8` et `colsample_bytree=0.8` (chaque arbre voit 80 % des lignes et des colonnes), jusqu'à 1 500 arbres, et **arrêt anticipé** : on s'arrête quand la **PR-AUC de validation** ne progresse plus pendant 30 arbres. L'entraînement s'arrête vers ~120 arbres.

> 🔧 **Choix de la métrique d'arrêt :** avec la *log-loss* comme critère, XGBoost allait jusqu'à ~900 arbres et sa PR-AUC de validation **baissait** (0,41 contre 0,46). La log-loss récompense la calibration globale, dominée par les 94 % de négatifs ; la PR-AUC mesure ce qui nous intéresse : bien classer les rares positifs.

7.5 Les réseaux récurrents : LSTM et GRU

Au lieu de recevoir l'historique sous forme de colonnes, un réseau récurrent **lit la séquence** des 10 dernières secondes (10 vecteurs de 83 variables), un pas après l'autre, en mettant à jour un **état caché** qui résume ce qu'il a vu.



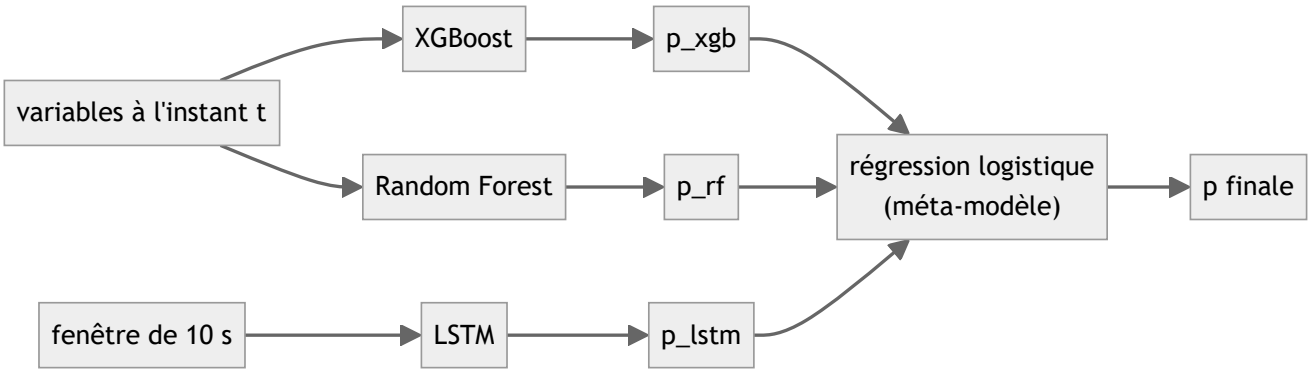
La cellule **LSTM** (*Long Short-Term Memory*) a trois **portes** apprises (oubli, entrée, sortie) qui décident quoi garder, quoi ajouter et quoi exposer de sa mémoire. Cela évite l'évanouissement du gradient des RNN simples. La **GRU** (*Gated Recurrent Unit*) est une version simplifiée à deux portes, avec moins de paramètres.

Architecture du projet : 2 couches de 64 unités, *dropout* 0,3, puis une couche dense de 32 neurones. Entraînement : Adam (lr = 10^{-3}), mini-lots de 256, réduction du lr sur plateau, **arrêt anticipé sur la PR-AUC de validation** (patience 5).

Les fenêtres sont construites UE par UE : une fenêtre ne mélange jamais deux terminaux. Les 9 premières secondes d'un UE sont complétées avec sa propre première mesure (`score_sequence_model`). En v3, l'inférence glissait la fenêtre sur tout le fichier : les 9 premières lignes de chaque UE utilisaient l'historique du terminal précédent.

> 💡 **Pourquoi les LSTM ne battent-ils pas les arbres ici ?** Parce que les variables de lag, delta et fenêtre glissante **donnent déjà la dynamique** aux modèles tabulaires. La fenêtre de 10 s n'apporte pas beaucoup plus d'information que ces résumés. Les LSTM brillent quand la dynamique utile est longue ou complexe, ou quand on ne sait pas quelles variables construire.

7.6 Le stacking



Le méta-modèle apprend **comment pondérer** les trois avis. Il est entraîné sur les prédictions de la **validation** (les modèles de base ne l'ont pas vue pour apprendre). Deux précautions :

- `class_weight="balanced"` sur le méta-modèle (sinon, avec 6 % de positifs, son seuil 0,5 est beaucoup trop haut : en v3 il ne détectait que 40 % des handovers) ;
- pour **comparer équitablement** le stacking aux autres modèles sur la validation, son score de validation est calculé **hors échantillon** (*out-of-fold*, validation croisée en 5 blocs temporels). Sinon il serait évalué sur les données mêmes qui ont servi à l'entraîner.

7.7 Traiter le déséquilibre des classes

Avec 6 % de positifs, un modèle naïf apprend à toujours répondre « non ». Chaque famille a son mécanisme :

Modèle	Mécanisme	Effet
Régression logistique, RF	`class_weight = {1: n_neg/n_pos}`	chaque positif compte ~15 fois plus dans la perte / le critère de division
XGBoost	`scale_pos_weight = n_neg/n_pos`	multiplie le gradient des positifs
LSTM, GRU	`WeightedRandomSampler`	les mini-lots contiennent ~50 % de positifs
Stacking	`class_weight="balanced"`	idem pour le méta-modèle

> 🦎 **Bug corrigé, à raconter :** en v3, les LSTM/GRU combinaient le *sampler* équilibré **et** un `pos_weight` dans la perte. Les positifs étaient donc pondérés deux fois, soit $(n_{neg}/n_{pos})^2 \approx 240\times$ au taux actuel de 6 %. Le réseau poussait toutes ses probabilités vers le haut : précision ≈ 0,33. Une seule correction suffit ; la v4 garde le *sampler* seul.

Le rééquilibrage déplace les probabilités : un modèle pondéré sur-estime la probabilité des positifs. Ses sorties ne sont plus des « vraies » probabilités, d'où l'importance de **choisir le seuil sur la validation** (chapitre 8) plutôt que de garder 0,5.

7.8 Reproductibilité

Toutes les sources d'aléa sont fixées : graine du simulateur (42), `random_state=42` pour scikit-learn et XGBoost, `torch.manual_seed`, générateur du *sampler*, algorithmes déterministes. **Deux exécutions complètes du pipeline (`dvc repro`) donnent des métriques identiques à la 4e décimale pour les sept modèles**, LSTM et GRU compris. En v3, ces derniers variaient d'environ ±0,015 de F1 d'une exécution à l'autre.

7.9 Ce qu'il faut retenir

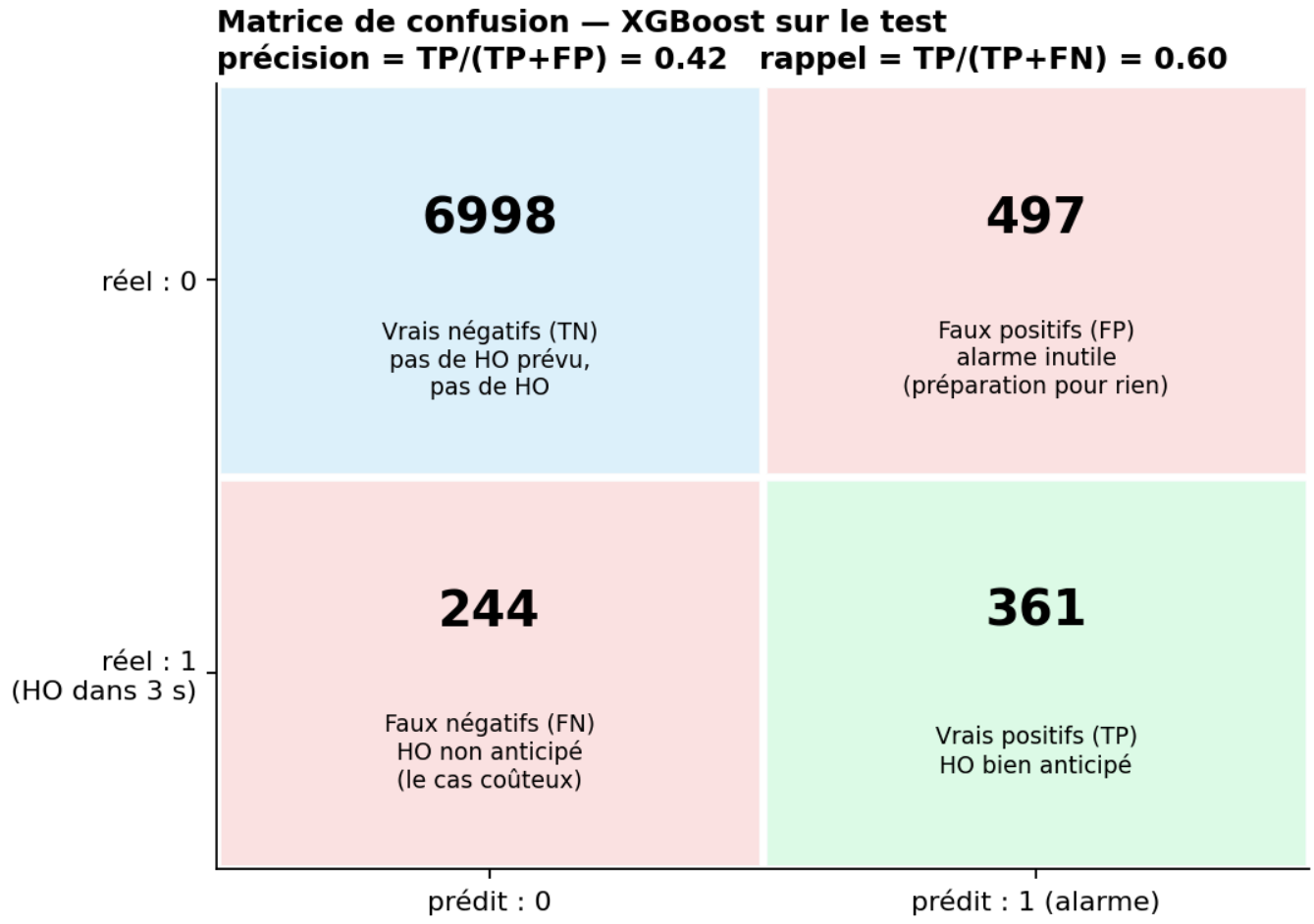
- Sept modèles de complexité croissante, dont une **règle sans apprentissage** comme référence.
- **Bagging** (RF) réduit la variance ; **boosting** (XGBoost) réduit le biais pas à pas ; **LSTM/GRU** lisent la séquence brute ; le **stacking** combine.
- Déséquilibre : **une** pondération par modèle (pas deux !), puis seuil choisi sur la validation.
- Arrêt anticipé sur la **PR-AUC**, adaptée aux classes rares.

Chapitre 8 — Évaluation et résultats

> **Objectif :** savoir lire et défendre chaque métrique, expliquer comment le seuil et le champion sont choisis sans tricher, et présenter les résultats, y compris ce que le modèle **ne** sait **pas** faire.

8.1 La matrice de confusion

Avec un seuil, chaque prédiction tombe dans l'une de quatre cases :

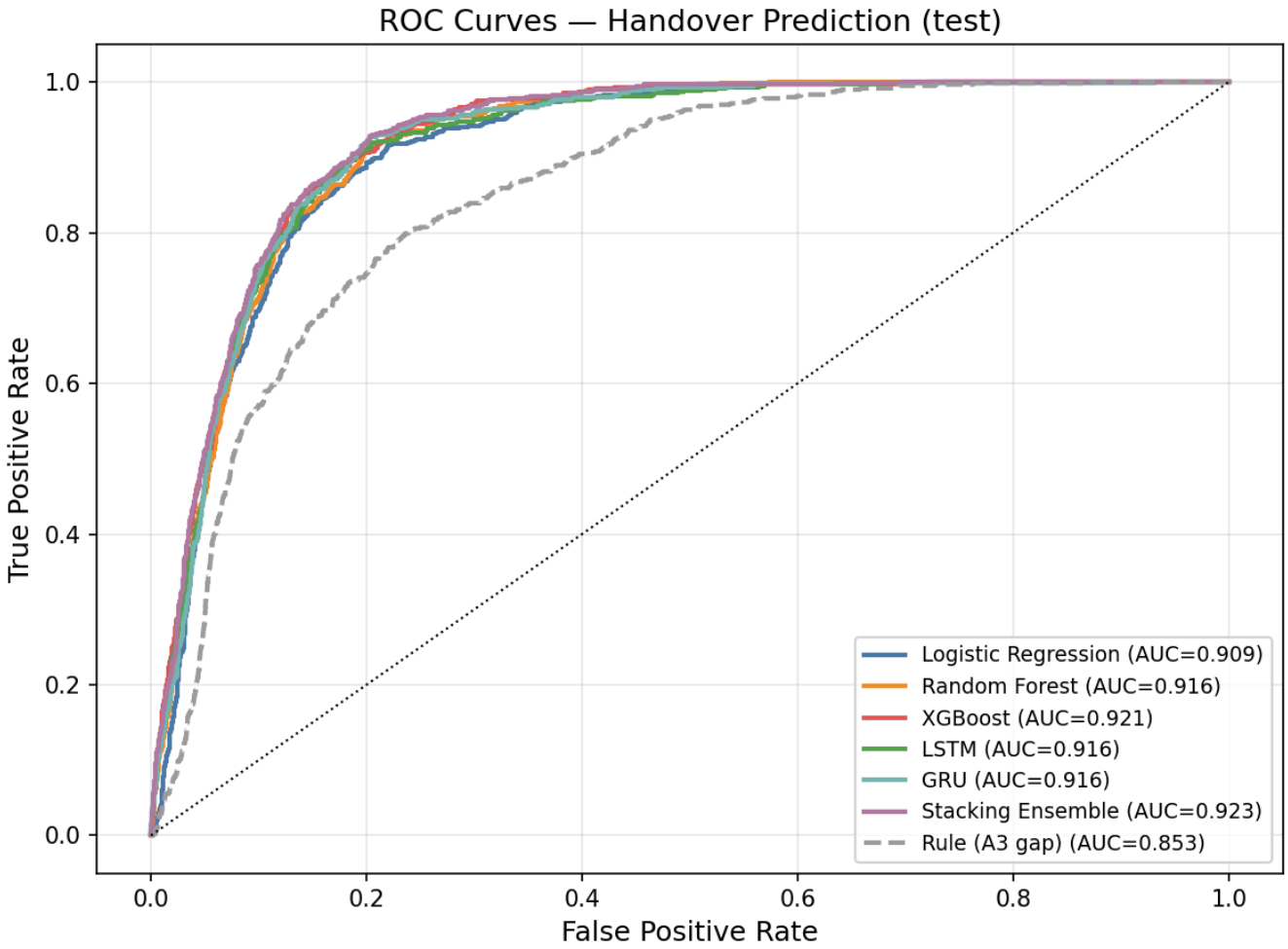


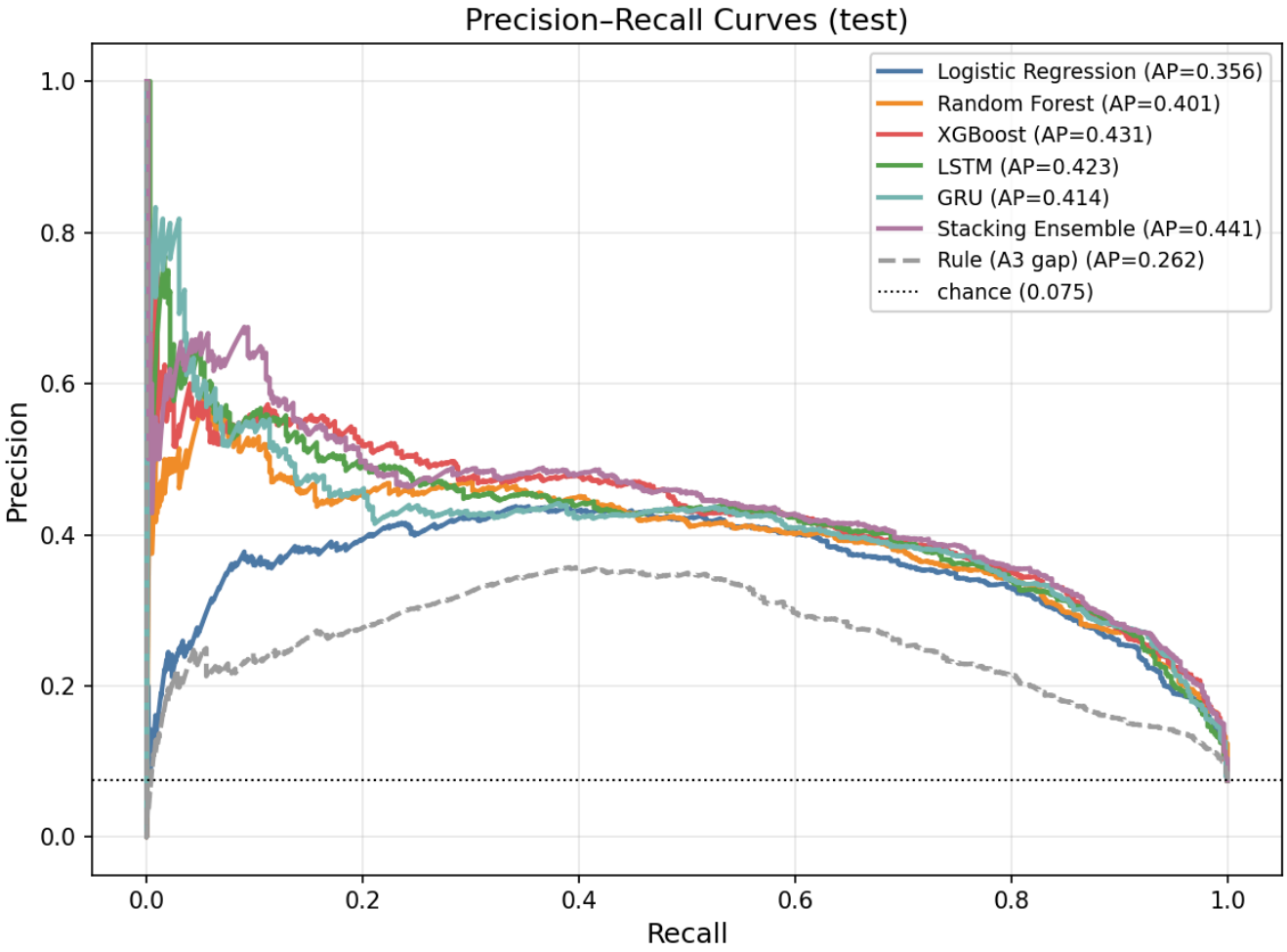
Métrique	Formule	Question à laquelle elle répond
Précision	TP / (TP + FP)	« Quand j'alarme, ai-je raison ? » (coût : préparations inutiles)
Rappel	TP / (TP + FN)	« Parmi les handovers à venir, combien ai-je vus ? » (coût : handovers non anticipés)
F1	2·P·R / (P + R)	moyenne harmonique : élevée seulement si **les deux** le sont
Accuracy	(TP + TN) / total	**trompeuse ici** : « toujours non » donne 92 %

8.2 ROC-AUC et PR-AUC

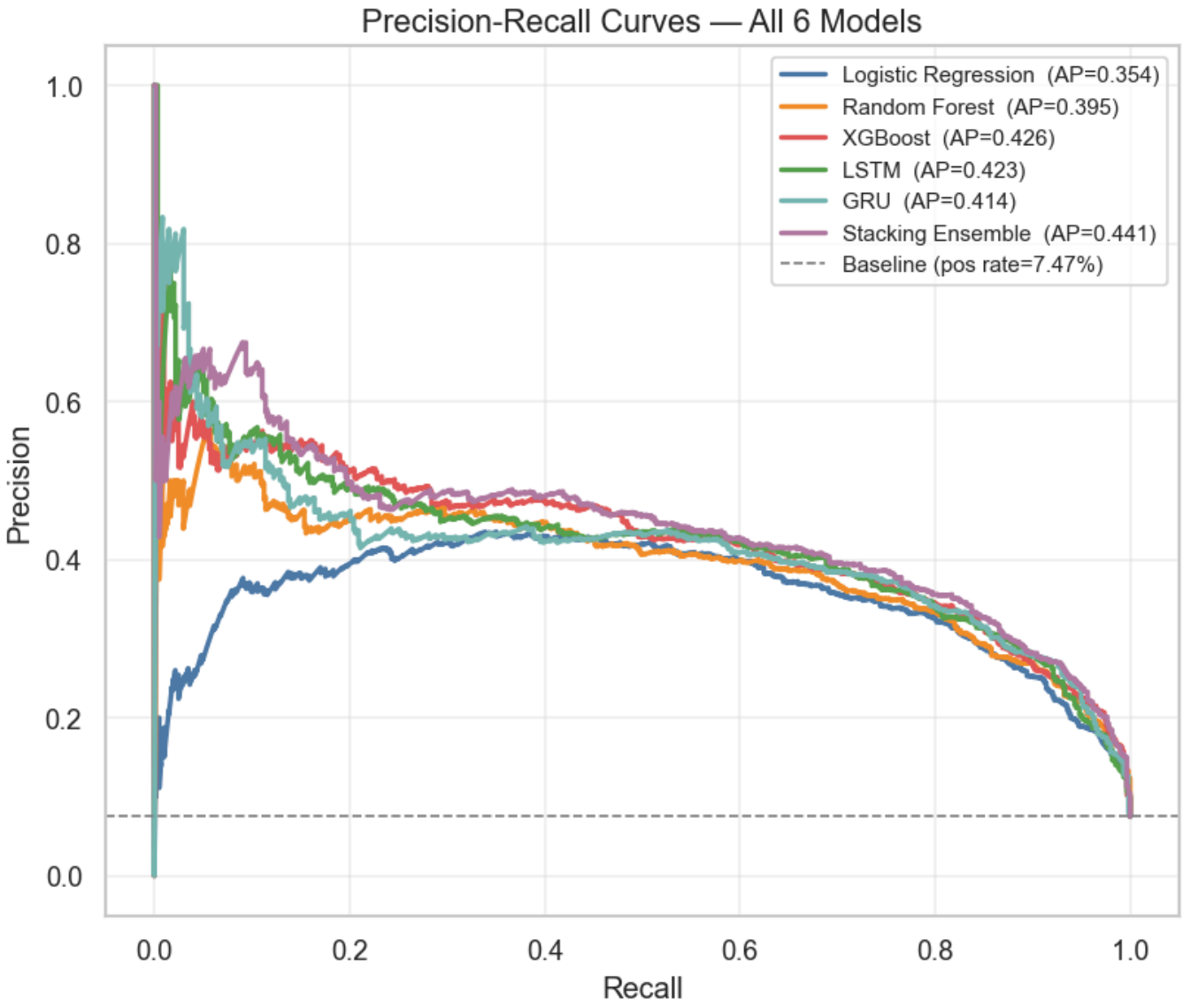
Ces deux métriques **ne dépendent pas du seuil** : elles mesurent la qualité du **classement** (les positifs ont-ils des scores plus élevés que les négatifs ?).

- **ROC-AUC** : aire sous la courbe taux de vrais positifs / taux de faux positifs. 0,5 = hasard, 1 = parfait. Interprétation : probabilité qu'un positif tiré au hasard ait un score supérieur à un négatif tiré au hasard.
- **PR-AUC** (*average precision*) : aire sous la courbe précision / rappel. Le hasard vaut le **taux de positifs** (7,5 % sur le test), et non 0,5.





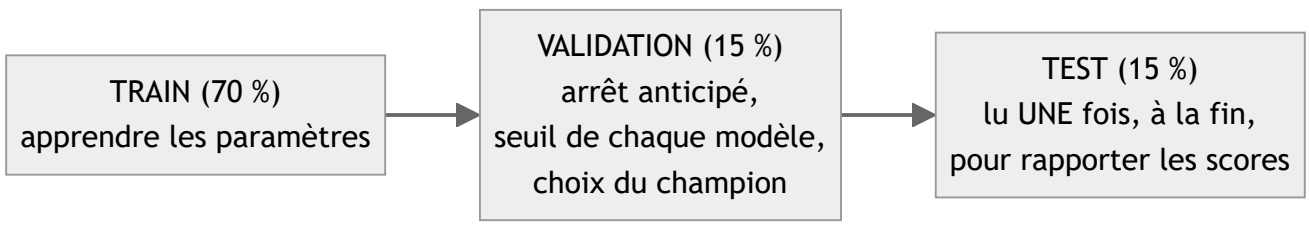
La même analyse dans le notebook 10, où la ligne pointillée marque le niveau du hasard (taux de positifs = 7,5 %) :



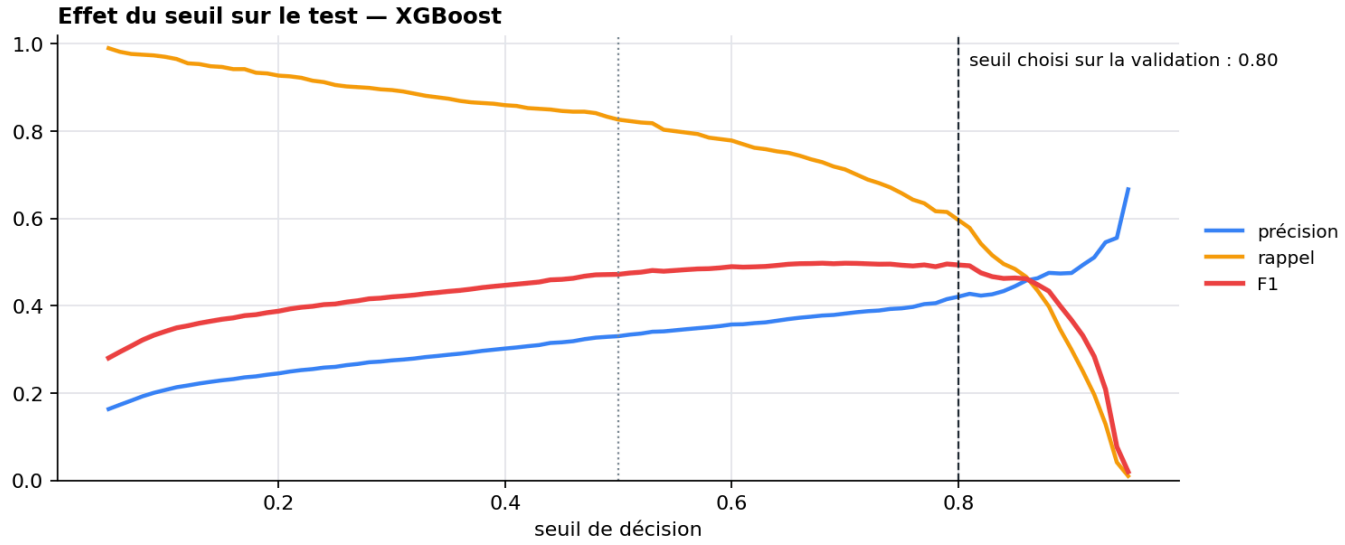
> 🎯 **Pourquoi regarder la PR-AUC quand les positifs sont rares ?** Le taux de faux positifs (axe de la ROC) a pour dénominateur les **négatifs**, très nombreux : 500 fausses alarmes sur 7 400 négatifs ne font que 7 %, et la courbe ROC reste flatteuse. La précision a pour dénominateur les **alarmes** : les mêmes 500 fausses alarmes pour 300 vraies donnent une précision de 37 %. La PR-AUC montre donc ce que vivra l'opérateur. C'est pour cela que les modèles plafonnent vers 0,92 de ROC-AUC mais 0,43 de PR-AUC.

8.3 Choisir le seuil, choisir le modèle, sans regarder le test

Le protocole est en trois temps, sur trois jeux de données disjoints dans le temps :



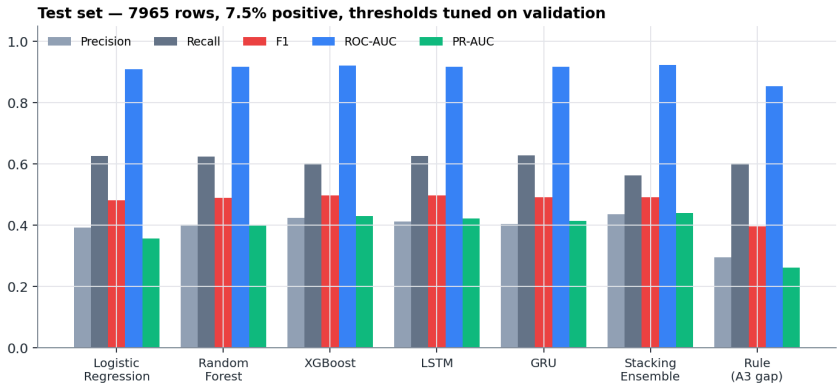
Le seuil. Les modèles pondérés (chapitre 7) ne sont pas calibrés : 0,5 n'a pas de sens particulier. Pour chaque modèle, on balaie les seuils de 0,05 à 0,95 sur la **validation** et on garde celui qui maximise le F1. Pour XGBoost, c'est **0,80**.

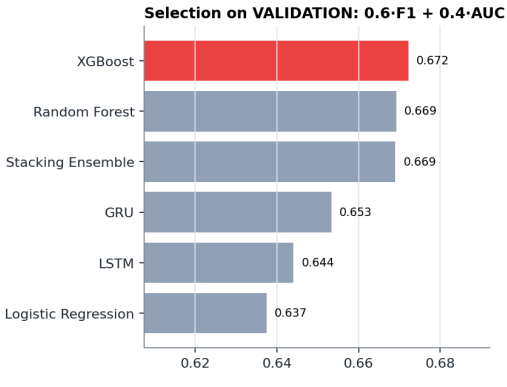


Le champion maximise $0{,}6 \times F1_{val} + 0{,}4 \times AUC_{val}$ (chapitre 10).

Tous les modèles sont évalués sur les mêmes lignes : les lignes de test qui ont 10 s d'historique, nécessaires aux LSTM. En v3, les modèles tabulaires étaient évalués sur plus de lignes que les LSTM, ce qui faussait légèrement la comparaison.

8.4 Les résultats





Validation (sert à choisir) :

Modèle	Seuil	F1	ROC-AUC	PR-AUC	Score de sélection
XGBoost ★	0,80	**0,502**	**0,927**	**0,471**	**0,672**
Random Forest	0,60	0,500	0,923	0,434	0,669
Stacking	0,89	0,502	0,920	0,432	0,669
GRU	0,81	0,479	0,915	0,411	0,653
LSTM	0,82	0,470	0,905	0,388	0,644
Régression logistique	0,78	0,457	0,909	0,322	0,637
Règle A3 (non éligible)	0,16	0,409	0,856	0,236	—

Test (rapporté une seule fois ; 7 965 lignes, 7,5 % de positifs) :

Modèle	Précision	Rappel	F1	ROC-AUC	PR-AUC
XGBoost ★	0,424	0,601	**0,497**	0,921	0,431
LSTM	0,413	0,627	0,498	0,916	0,423
GRU	0,405	0,628	0,492	0,916	0,414
Stacking	0,435	0,563	0,491	**0,923**	**0,441**
Random Forest	0,403	0,623	0,489	0,916	0,401
Régression logistique	0,392	0,627	0,482	0,909	0,356
Règle A3	0,296	0,603	**0,397**	**0,853**	**0,262**

Comment les présenter

1. **Le ML apporte un gain net sur la règle** : +0,10 de F1, +0,07 de ROC-AUC, PR-AUC multipliée par ~1,6. À rappel égal (~60 %), la règle donne 30 % de précision contre 42 % pour XGBoost : beaucoup moins d'alarmes inutiles.
2. **Les modèles appris sont très proches entre eux** (F1 0,48–0,50 sur le test). Les écarts sont de l'ordre du bruit statistique. Le choix du champion se joue à 0,003 près sur la validation : en entretien, ne surinterprétez pas le classement.
3. **Un F1 de 0,5 est un résultat honnête, pas un échec.** Le label demande de prédire un événement **3 s à l'avance**, dans un canal partiellement aléatoire (masquage, transitions LOS). Une partie des handovers n'a simplement **pas de signe avant-coureur** 3 s avant. Un AUC de 0,92 montre que le classement est bon ; le plafond du F1 vient du déséquilibre et du bruit du label.

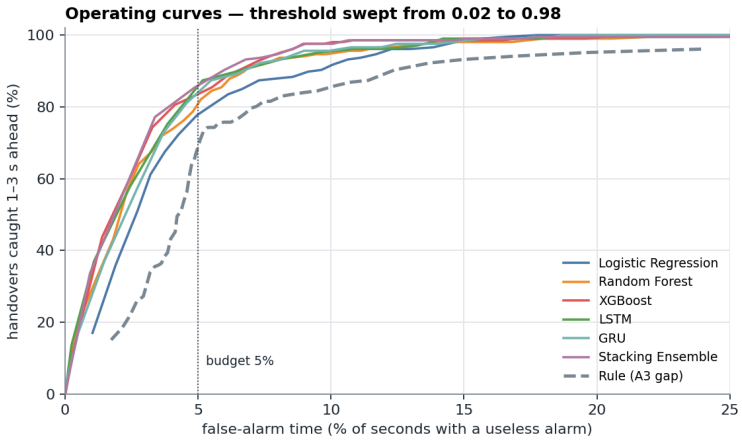
8.5 L'évaluation opérationnelle : penser en événements

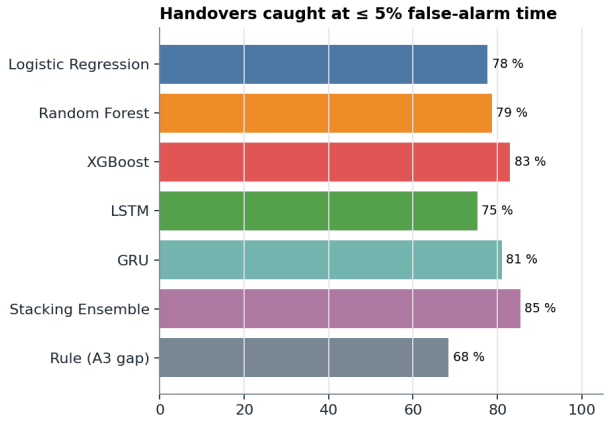
Un opérateur ne raisonne pas en secondes bien classées, mais en **handovers anticipés** et en **coût des alarmes**. Trois métriques (`src/operational.py`) :

- **rappel événementiel à 3 s** : part des handovers réussis précédés d'au moins une alarme dans les 3 s ;
- **délai d'anticipation** : quand arrive la première alarme ;
- **temps de fausse alarme** : part des secondes où une alarme n'est suivie d'aucun handover dans les 3 s, c'est-à-dire le temps passé à préparer un handover pour rien.

> 🦎 **Piège évité :** une première version comptait les fausses alarmes par *épisode* (suite de secondes d'alarme consécutives). Un modèle qui alarme **en permanence** ne produit alors qu'un seul épisode, donc « presque aucune fausse alarme » ! Le stacking obtenait ainsi un rappel de 100 % sous un budget de fausses alarmes très bas. Mesurer le coût **en temps** corrige ce biais (un test le vérifie).

Comparer les modèles à leur seuil respectif mélange deux choses. On trace donc des **courbes opérationnelles** (rappel événementiel en fonction du temps de fausse alarme, seuil balayé) et on compare **à budget égal** :





Modèle	Handovers anticipés à ≤ 5 % de temps de fausse alarme
Stacking	**85 %**
XGBoost ★	**83 %**
GRU	81 %
Random Forest	79 %
Régression logistique	78 %
LSTM	75 %
Règle A3	**68 %**

Au seuil choisi, XGBoost anticipe **89 % des handovers** (91 % dans les 5 s), avec une première alarme **3,4 s** avant en moyenne, pour 6,1 % de temps de fausse alarme.

> 🎯 **Phrase à retenir :** « Pour un même coût d'alarmes inutiles (5 % du temps), mon modèle anticipe 83 % des handovers contre 68 % pour la règle A3. »

8.6 Explicabilité : SHAP

SHAP (*SHapley Additive exPlanations*) décompose chaque prédiction en contributions additives des variables, selon la théorie des jeux (valeurs de Shapley) :

$$f(x) = \phi_0 + \sum_j \phi_j(x)$$

ϕ_0 est la prédiction moyenne et ϕ_j la contribution de la variable j pour cet exemple. Pour les arbres, `TreeExplainer` calcule ces valeurs exactement et rapidement.

Importance globale pour le champion XGBoost (moyenne de |φ| sur le test) :

Rang	Variable	Part de l'attribution
1	`ue_speed` (vitesse)	17,7 %
2	`rsrp_diff` (RSRP service − voisine)	14,9 %
3	`sinr`	9,8 %
4	`rsrq_serving`	8,7 %
5–7	`rsrp_neighbor`, `rsrq_neighbor`, `rsrp_serving`	3,2 / 3,1 / 2,1 %
8+	`sinr_lag1`, `rsrq_serving_roll3_mean`, `rsrp_neighbor_lag1`, `rsrp_neighbor_roll3_std`, `cqi` …	≤ 1,8 % chacune

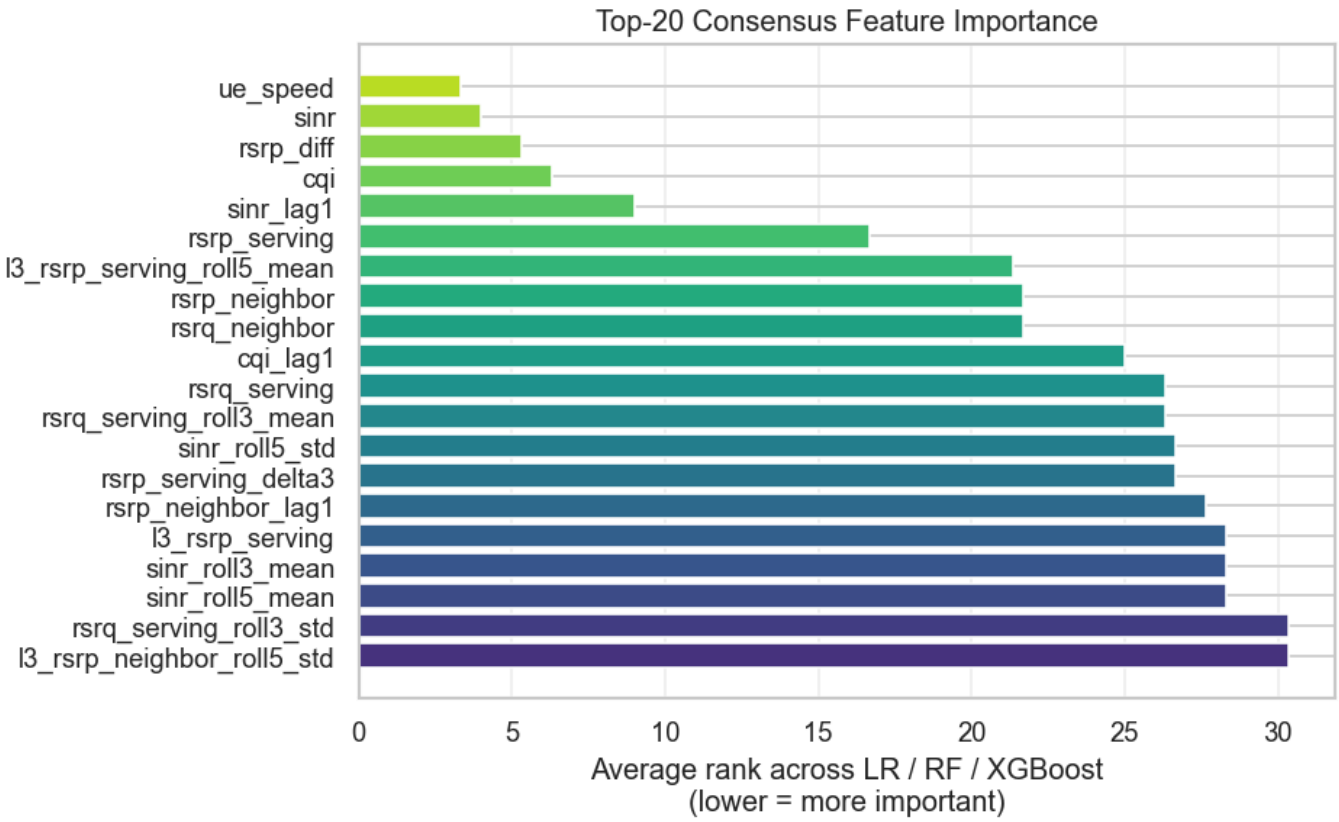
Par famille : **valeurs courantes et contexte ≈ 63 %**, moyennes/écarts-types glissants ≈ 20 %, retards ≈ 10 %, variations ≈ 6 %.

Comment l'interpréter :

- `rsrp_diff` est l'écart qui pilote A3. Sa contribution est fortement **négative quand l'écart est positif** (service nettement meilleure → pas de handover ; corrélation valeur/SHAP de −0,84). Quand la voisine rattrape la cellule de service, la contribution devient positive. C'est la physique d'A3, retrouvée par le modèle.
- **SINR et RSRQ** baissent à l'approche de la frontière (chapitre 2) : ils indiquent la proximité du bord de cellule.
- **La vitesse arrive en tête**, et c'est logique : les véhicules changent de cellule ~4 fois plus souvent que les piétons, et leur TTT est plus court. La vitesse sert de **probabilité a priori** (« cet UE fait beaucoup de handovers »), que les variables radio affinent ensuite.
- **Les variables temporelles** pèsent ~37 % au total : aucune n'est dominante seule, mais ensemble elles apportent la **tendance** (la voisine monte-t-elle ? le lien est-il instable ?).

> ⚠️ **À dire honnêtement :** le modèle s'appuie d'abord sur l'**état courant** (écart service/voisine, SINR, RSRQ) et sur la vitesse ; la dynamique **complète** cette information plutôt qu'elle ne la remplace. C'est aussi pourquoi la règle A3, qui ne voit que l'écart instantané, fait déjà une partie du travail.

Le notebook 09 croise les importances des trois modèles explicables (rang moyen sur régression logistique, forêt et XGBoost) : vitesse, SINR, `rsrp_diff` et CQI arrivent en tête pour tous, ce qui montre que ce constat ne dépend pas du modèle.



Les graphiques de SHAP pour tous les modèles sont dans `reports/shap/` et dans l'onglet SHAP du dashboard ; le notebook 09 les analyse en détail.

8.7 Ce qu'il faut retenir

- Positifs rares → **F1 et PR-AUC**, pas l'accuracy. ROC-AUC élevée ≠ peu de fausses alarmes.
- **Seuil et champion choisis sur la validation**, test lu une fois.
- Champion **XGBoost** : test F1 0,50, ROC-AUC 0,92, PR-AUC 0,43, contre 0,40 / 0,85 / 0,26 pour la règle A3.
- En événements : **83 % des handovers anticipés** à 5 % de temps de fausse alarme (68 % pour la règle), ~3,4 s d'avance.
- SHAP : le modèle s'appuie d'abord sur l'**écart service/voisine, le SINR, le RSRQ et la vitesse** ; les tendances apportent ~37 % de l'attribution.

Chapitre 9 — Boucle fermée et généralisation

> **Objectif :** répondre aux deux questions qu'un responsable technique posera forcément. **« Est-ce que votre prédiction améliore vraiment le réseau ? »** et **« Est-ce que ça marche sur un réseau que le modèle n'a jamais vu ? »**

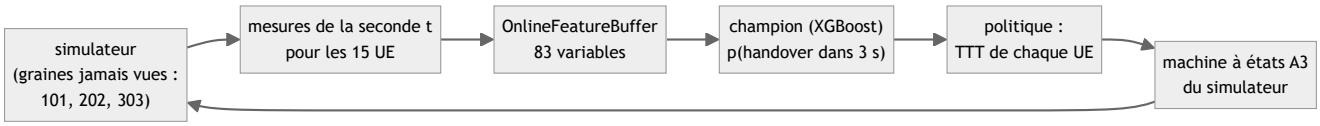
9.1 De la prédiction à l'action

Un bon score de classification ne prouve pas qu'on améliore le réseau. La prédiction n'a de valeur que si on **agit** dessus. L'action la plus naturelle est d'adapter le **TTT** de chaque UE (chapitre 4) :

- si le modèle est **confiant qu'un handover arrive**, on raccourcit le TTT : le handover part plus tôt, quand le SINR est encore bon, donc il échoue moins ;
- si la condition A3 est vraie mais que le modèle pense que **ce n'est pas un vrai handover** (fluctuation passagère), on allonge le TTT pour éviter un ping-pong.

C'est une forme de **MRO prédictif**, UE par UE et seconde par seconde.

9.2 Le protocole expérimental



Cinq politiques sont comparées, sur **3 graines jamais vues × 1 800 s** :

Politique	Règle	Rôle
`baseline`	TTT selon la vitesse (comme dans les données d'entraînement)	référence
`always_short`	TTT = 1 s pour tout le monde	réagir vite partout
`always_long`	TTT = 4 s pour tout le monde	filtrer partout
`predictive`	p ≥ 0,80 → TTT 1 s ; A3 vrai et p < 0,17 → TTT + 2 s ; sinon TTT normal	**la proposition**
`random_gate`	mêmes modifications, **mêmes taux**, mais à des instants tirés au hasard	**témoin**

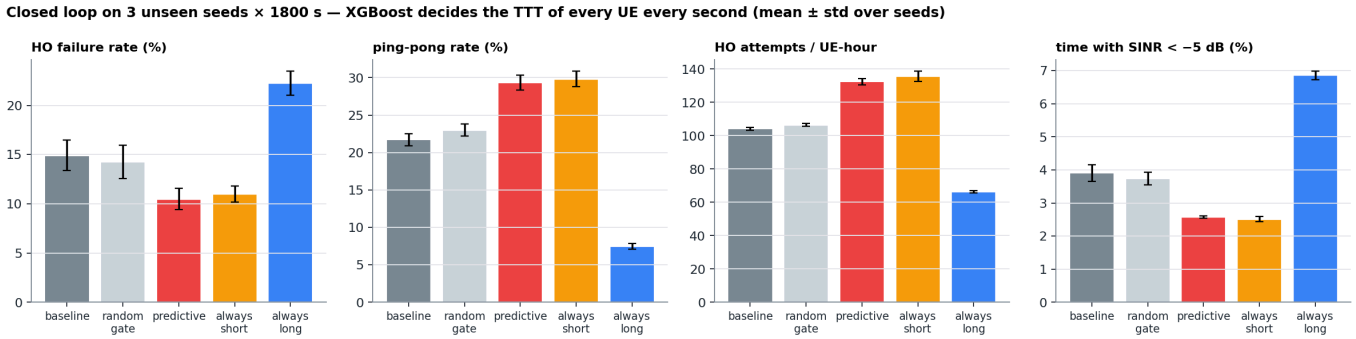
Les deux seuils ne sont pas réglés sur ces graines. Le seuil haut (0,80) est celui du champion choisi sur la validation ; le seuil bas (0,17) est le 5ᵉ centile des probabilités du champion sur les vrais positifs de la validation : seuls 5 % des vrais handovers seraient retenus.

Pourquoi c'est rigoureux :

- grâce aux **deux générateurs aléatoires** du simulateur, les 5 politiques voient **exactement les mêmes trajectoires et le même canal radio** pour une graine donnée : seules les décisions de handover diffèrent ;

- le **témoin aléatoire** applique les mêmes modifications avec la même fréquence (TTT raccourci sur ~9,5 % des UE-secondes, allongé sur 5 à 7 % des secondes où A3 est vrai). Si `predictive` fait mieux que `random_gate`, c'est que **le modèle choisit les bons moments**, pas seulement qu'on modifie le TTT plus souvent.

9.3 Les résultats



Politique	Échecs de HO	Ping-pong	Secondes en RLF / UE·h	SINR à l'exécution	Temps avec SINR < −5 dB	Tentatives / UE·h
baseline	14,9 %	21,7 %	62,1	−6,2 dB	3,9 %	104
random_gate	14,2 %	23,0 %	60,6	−5,9 dB	3,7 %	106
predictive	**10,5 %**	29,4 %	**55,5**	**−4,1 dB**	**2,6 %**	132
always_short	11,0 %	29,8 %	59,6	−4,0 dB	2,5 %	136
always_long	22,3 %	7,5 %	59,1	−10,1 dB	6,9 %	66

(moyennes sur 3 graines ; écarts-types dans `reports/closed_loop.json`)

Comment les lire

1. La prédiction choisit les bons moments. À fréquence de modification égale, le témoin aléatoire ne change presque rien par rapport à la référence (14,2 % d'échecs contre 14,9 %). La politique prédictive fait baisser les échecs de **14,9 % à 10,5 %** (−30 % relatifs), remonte le SINR au moment du handover de **2 dB**, et réduit d'un tiers le temps passé avec un SINR inférieur à −5 dB.

2. Mais elle fait à peine mieux qu'une règle simple. `always_short` obtient presque les mêmes gains (11,0 % d'échecs). La prédictive reste légèrement meilleure sur les quatre indicateurs (échecs, temps en RLF, ping-pong, nombre de tentatives), mais ces écarts sont de l'ordre d'un écart-type entre graines : **ils ne sont pas significatifs** avec 3 graines. Pourquoi si peu de différence ? Le modèle est surtout confiant **quand A3 est déjà vrai ou presque** : raccourcir le TTT « au bon moment » ressemble beaucoup à le raccourcir « toujours ».

3. Le compromis de la mobilité apparaît clairement. Avancer les handovers réduit les échecs mais **augmente les ping-pongs** (21,7 % → 29,4 %). `always_long` fait exactement l'inverse : 7,5 % de ping-pong, mais 22,3 % d'échecs et deux fois plus de temps en mauvais SINR. Aucune politique ne gagne sur les deux tableaux. C'est le dilemme « trop tardif contre ping-pong » du chapitre 4.

> 🎯 **Conclusion à donner en entretien :** « J'ai mis le modèle dans la boucle de décision du simulateur. À fréquence d'intervention égale, il réduit les échecs de handover de 30 % là où un témoin aléatoire ne fait rien : il détecte donc les bons moments. Mais une simple politique de TTT court fait presque aussi bien, et les deux augmentent les ping-pongs. Prédire **qu'un** handover arrive ne suffit pas : pour battre le compromis, il faudrait prédire **si ce handover va être un ping-pong**. C'est la suite logique du projet. »

Cette conclusion honnête vaut plus qu'un résultat flatteur : elle montre que vous savez concevoir un protocole qui peut **infirmer** votre hypothèse.

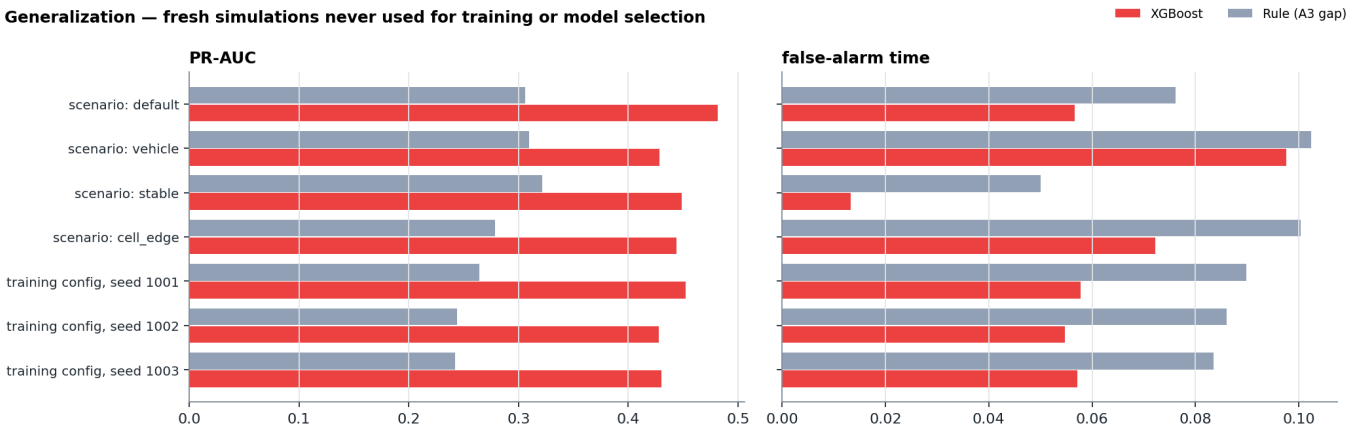
Limite à mentionner

Le modèle a été entraîné sur des données produites avec la politique `baseline`. Quand la politique change l'instant des handovers, la distribution des situations que voit le modèle **dérive légèrement** (*distribution shift*). Une version plus poussée ré-entraînerait le modèle sur des données collectées sous la nouvelle politique (apprentissage itératif, ou apprentissage par renforcement).

9.4 Généralisation à des réseaux jamais vus

Le test du chapitre 8 porte sur la même simulation que l'entraînement (les 15 % dernières secondes). On génère donc des simulations **entièrement nouvelles**, jamais utilisées ni pour l'entraînement ni pour choisir le modèle :

- les 4 **scénarios** de `generate_test_dataset.py` (8 UE × 1 800 s chacun) : `default` (mixte), `vehicle` (que des véhicules rapides), `stable` (piétons lents près des antennes), `cell_edge` (UE semés le long des frontières) ;
- **3 graines inédites** de la configuration d'entraînement (15 UE × 1 800 s).



Jeu de données	Taux de positifs	AUC champion	PR-AUC champion	PR-AUC règle	Temps de fausse alarme (champion / règle)
config. d'entraînement, graine 1001	7,1 %	0,929	0,452	0,264	5,8 % / 9,0 %
config. d'entraînement, graine 1002	6,5 %	0,929	0,428	0,244	5,5 % / 8,6 %
config. d'entraînement, graine 1003	6,4 %	0,930	0,430	0,242	5,7 % / 8,4 %
scénario default	7,2 %	0,931	0,482	0,306	5,7 % / 7,6 %
scénario cell_edge	8,3 %	0,908	0,444	0,278	7,2 % / 10,0 %
scénario vehicle	11,7 %	0,873	0,429	0,310	9,8 % / 10,2 %
scénario stable	1,5 %	0,977	0,449	0,322	1,3 % / 5,0 %

Lecture :

- Sur des graines inédites, le champion garde **AUC ≈ 0,93 et PR-AUC ≈ 0,43–0,45**, le niveau du test : **pas de sur-apprentissage** à la simulation d'entraînement.
- Le scénario **véhicules** est le plus difficile (AUC 0,87) : à haute vitesse, la fenêtre entre « signes avant-coureurs » et handover est très courte (TTT de 1 s).
- Le champion bat la règle **partout**, en PR-AUC comme en temps de fausse alarme.

9.5 Ce qu'il faut retenir

- Mettre le modèle **dans la boucle** est la seule façon de mesurer un gain réseau.
- Protocole : **mêmes trajectoires** pour toutes les politiques + **témoin aléatoire** à taux égal.
- Résultat : −30 % d'échecs de handover par rapport à la référence, là où le témoin ne fait rien. Gain **équivalent** à un TTT court, au prix de plus de ping-pongs : c'est le compromis de la mobilité.
- Suite logique : **prédire les ping-pongs**, ré-entraîner sous la nouvelle politique.
- Généralisation : performances stables sur des réseaux inédits ; les véhicules rapides sont le cas difficile.

Chapitre 10 — MLOps, tests et organisation du code

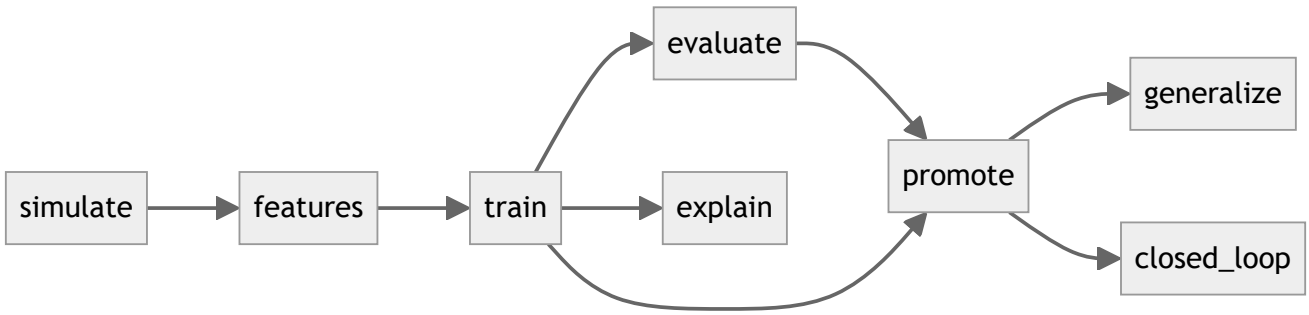
> **Objectif :** savoir expliquer comment le projet est industrialisé (reproductibilité, suivi des expériences, intégration continue, tests, mise à disposition du modèle), et où se trouve chaque chose dans le code.

10.1 Pourquoi du MLOps sur un projet de simulation ?

Un résultat de ML n'a de valeur que s'il est **reproductible** : quelqu'un d'autre doit pouvoir relancer le pipeline et retrouver les mêmes chiffres. Le projet s'appuie sur trois outils standards, plus des tests :

Besoin	Outil	Ce qu'il apporte
rejouer le pipeline sans tout recalculer	**DVC**	graphe d'étapes avec dépendances ; ne relance que ce qui a changé
garder la trace des expériences	**MLflow**	hyperparamètres, métriques, modèles et graphiques de chaque entraînement
vérifier chaque modification	**GitHub Actions**	à chaque *push*, tests + pipeline complet + publication des rapports
garantir les propriétés critiques	**pytest**	36 tests (physique radio, logique de handover, anti-fuite, inférence)

10.2 Le pipeline DVC



Chaque étape de `dvc.yaml` déclare sa **commande**, ses **dépendances** (code et données) et ses **sorties**. `dvc repro` compare les empreintes (hash) des dépendances avec celles enregistrées dans `dvc.lock`, et ne relance que les étapes périmées. Modifier `src/models.py` relance l'entraînement et ce qui en découle, mais pas la simulation.

Étape	Commande	Produit
simulate	`python simulate.py`	`data/raw/dataset.csv`
features	`python run_pipeline.py --phase 2`	train / val / test, `scaler.pkl`, `meta.json`
train	`--phase 3`	7 modèles, `thresholds.json` (seuils et métriques de validation)
evaluate	`--phase 4`	`evaluation.json`, `operational.json`, courbes ROC/PR, matrices de confusion
explain	`--phase 5`	graphiques SHAP
promote	`scripts/promote_best_model.py`	`models/champion/`
generalize	`python -m src.generalization`	`generalization.json`
closed_loop	`python -m src.closed_loop`	`closed_loop.json`

Les rapports JSON sont versionnés dans Git (`cache: false`) pour que les chiffres du README soient vérifiables. Les modèles et les CSV volumineux sont régénérés.

10.3 MLflow : le journal des expériences

Chaque entraînement crée un *run* MLflow (base SQLite locale `mlflow.db`, sans serveur) contenant : hyperparamètres, métriques de **validation** (F1 au seuil choisi, ROC-AUC, PR-AUC, seuil), modèle sérialisé. L'évaluation **reprend le même run** pour y ajouter les métriques de **test** et les graphiques. On peut ainsi comparer les modèles dans l'interface (`mlflow ui --backend-store-uri sqlite:///mlflow.db`) ou dans l'onglet du dashboard.

10.4 La sélection du champion

`scripts/promote_best_model.py` choisit le modèle qui maximise, **sur la validation** :

$$score = 0{,}6 \times F1_{val} + 0{,}4 \times AUC_{val}$$

puis le copie dans `models/champion/` avec un `metadata.json` (nom, seuil, métriques de validation et de test). Le F1 pèse davantage parce qu'il reflète le **point de fonctionnement réel** (avec le seuil choisi) ; l'AUC garde un œil sur la qualité du classement à tous les seuils. La règle sans apprentissage est évaluée mais **n'est pas éligible**.

> ⚠️ En v3, la sélection se faisait sur le **test**. C'est une erreur méthodologique : le score de test du champion devient optimiste, puisqu'on a choisi le meilleur sur ces mêmes données. La v4 sélectionne sur la validation et ne lit le test qu'ensuite.

10.5 L'intégration continue (GitHub Actions)

À chaque *push* ou *pull request* sur `main` :

1. **job** `tests` : installation, puis `pytest -q` (≈ 10 s) ;

2. **job** `pipeline` (seulement si les tests passent) : `dvc repro` complet (≈ 7 min), puis publication des rapports et des métadonnées du champion comme *artefacts*.

Un cache GitHub du *run-cache* DVC évite de tout recalculer quand le code n'a pas changé.

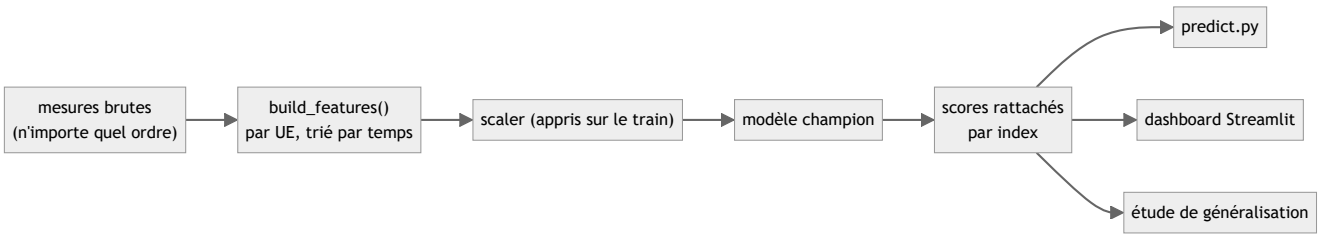
10.6 Les tests

Fichier	Ce qu'il garantit
`test_radio_model.py`	probabilité LOS décroissante, NLOS ≥ LOS, CQI monotone, RSRP dans [−140, −44], **SINR inchangé par le passage au RSRP par RE**, LOS fixe pour un UE immobile, moyennage du fast fading
`test_handover_logic.py`	TTT en fonction de la vitesse, déclenchement après exactement TTT pas, remise à zéro, changement de candidate, hystérésis après ping-pong, RLF après échec
`test_simulator.py`	même graine → mêmes données ; **une politique ne change pas les trajectoires** ; label correct ; RLF de 3 s exactement ; vitesse nulle à l'arrêt
`test_features.py`	83 variables sans position ni LOS ; **le futur ne modifie pas le passé** ; ordre des lignes sans effet ; parité en ligne / hors ligne
`test_serving.py`	un CSV brut est noté de bout en bout ; **mélanger les lignes ne change pas les scores** ; les fenêtres LSTM ne traversent jamais deux UE
`test_operational.py`	rappel événementiel, délai d'anticipation, coût des fausses alarmes (une alarme permanente doit coûter cher)

> 💡 Plusieurs tests **reproduisent un bug de la v3** : ils auraient échoué sur l'ancien code. C'est la bonne manière de figer une correction.

10.7 La mise à disposition du modèle

Un seul chemin d'inférence, `src/serving.py`, est partagé par tous les consommateurs :



- `predict.py` : `python predict.py --csv mesures.csv` affiche pour chaque ligne la probabilité et la décision au seuil du champion ; `--info` résume le champion.
- **Dashboard Streamlit** (`streamlit run app/dashboard.py`), 9 onglets : KPI radio, chronologie des handovers, carte de risque, prédiction en direct (dépôt d'un CSV), comparaison des modèles (avec métriques événementielles), SHAP, runs MLflow, carte de mobilité, et **Proactive HO** (boucle fermée et généralisation).

10.8 Carte du code

```
├── simulate.py                simulation d'entraînement (wrapper)
├── generate_test_dataset.py   scénarios de test (default, vehicle, stable, cell_edge)
├── run_pipeline.py            phases 2 à 5
├── predict.py                 inférence en ligne de commande
├── src/
│   ├── radio_model.py         propagation, RSRP par RE, SINR, RSRQ, CQI
│   ├── mobility.py            Random Waypoint, vitesse réelle
│   ├── handover_logic.py      filtrage L3, A3 + TTT, échec, ping-pong
│   ├── simulator.py           boucle de simulation partagée, RLF, label
│   ├── features.py            83 variables, build_features, OnlineFeatureBuffer
│   ├── models.py              7 modèles, seuils, scoring séquentiel par UE
│   ├── evaluate.py            métriques sur des lignes communes, courbes
│   ├── operational.py         métriques événementielles
│   ├── explain.py             SHAP
│   ├── serving.py             chemin d'inférence unique
│   ├── closed_loop.py         expérience en boucle fermée
│   └── generalization.py      généralisation
├── scripts/                   promotion du champion, génération des figures
├── app/dashboard.py           dashboard Streamlit
├── tests/                     36 tests pytest
├── notebooks/                 10 notebooks d'analyse
└── docs/                      README technique, rapport, ce cours
```

Chapitre 11 — Préparer l'entretien

> **Objectif :** être capable de présenter le projet en 2 minutes, de citer les chiffres clés sans hésiter, et de répondre aux questions techniques probables, en télécom comme en ML.

11.1 Le pitch en 2 minutes

> « Dans un réseau LTE, quand un utilisateur passe d'une cellule à l'autre, l'eNodeB déclenche un **handover** sur la base des mesures radio que le terminal lui remonte : principalement le **RSRP**, comparé entre la cellule de service et la voisine (événement **A3**). Si le handover est déclenché trop tard, le lien casse avant la fin de la procédure : c'est une **coupure**.
>
> Mon projet prédit, **3 secondes à l'avance**, qu'un handover réussi va avoir lieu, à partir uniquement des mesures standardisées que l'eNodeB reçoit déjà : RSRP, RSRQ, SINR, CQI, leurs tendances, la vitesse et la charge.
>
> Comme il n'existe pas de jeu de données public, j'ai écrit un **simulateur LTE** fondé sur les modèles 3GPP : propagation UMa, LOS/NLOS spatialement cohérent, masquage, évanouissement rapide, RSRP par élément de ressource, filtrage L3, A3 avec TTT, échecs, ping-pong. J'ai entraîné et comparé **six modèles et une règle de référence** (régression logistique, forêt aléatoire, XGBoost, LSTM, GRU, stacking), avec un découpage temporel, des seuils et une sélection faits sur la validation, dans un pipeline reproductible (DVC, MLflow, CI, 36 tests).
>
> Le champion, XGBoost, atteint une **ROC-AUC de 0,92** et anticipe **83 % des handovers à coût de fausses alarmes égal**, contre 68 % pour la règle A3. Enfin, je l'ai mis **dans la boucle de décision** du simulateur : il réduit les échecs de handover de **30 %**, là où un témoin aléatoire ne fait rien. Mais un simple TTT court fait presque aussi bien, et les deux augmentent les ping-pongs. Prédire le handover ne suffit donc pas à battre ce compromis : il faudrait prédire les ping-pongs. »

11.2 Les chiffres clés

Sujet	Chiffre
Réseau	4 eNodeB, 1 km², ISD 500 m, 2 GHz, 10 MHz, 46 dBm (18,2 dBm/RE)
UE	15 (8 piétons, 7 véhicules), 3 600 s, 54 000 lignes
Label	handover **réussi** dans les **3 s** ; **6,4 %** de positifs
Variables	**83** (8 signaux × 10 transformations + vitesse, rsrp_diff, charge)
Découpage	temporel 70 / 15 / 15
Handovers	1 434 tentatives / h ; 16 % d'échecs ; 20 % de ping-pong
Champion	**XGBoost**, seuil 0,80 (choisi sur validation)
Test	F1 **0,50**, ROC-AUC **0,92**, PR-AUC **0,43** (règle A3 : 0,40 / 0,85 / 0,26)
Événements	**89 %** des handovers anticipés, **3,4 s** d'avance en moyenne
À budget égal	**83 %** (ML) contre **68 %** (règle) à ≤ 5 % de temps de fausse alarme
Boucle fermée	échecs **14,9 %** → **10,5 %** ; témoin aléatoire 14,2 % ; TTT court 11,0 % ; ping-pong 21,7 % → 29,4 %
Généralisation	graines inédites : AUC 0,93, PR-AUC 0,43–0,45 ; véhicules : AUC 0,87
Reproductibilité	2 exécutions complètes → métriques identiques ; pipeline en ~7 min

11.3 Questions télécom

1. Qu'est-ce qu'un handover et pourquoi est-il critique ? Le transfert d'une communication en cours vers une autre cellule. En LTE il est « dur » (*break-before-make*) : s'il échoue, le lien radio casse (RLF) et la communication est coupée le temps du rétablissement.

2. Qui décide du handover en LTE ? L'eNodeB source, sur la base des *Measurement Reports* RRC de l'UE. L'UE mesure, le réseau décide. En veille (RRC_IDLE), c'est l'UE qui fait de la resélection, sans handover.

3. Décrivez la procédure de handover X2. Mesure et filtrage L3 → événement A3 vrai pendant le TTT → *Measurement Report* → décision de la source → *X2 Handover Request* → contrôle d'admission de la cible → *Request Ack* → *RRC Reconfiguration* (ordre de HO) → l'UE fait un RACH sur la cible → *Reconfiguration Complete* → *Path Switch* vers le MME/S-GW → libération du contexte dans la source.

4. Différence entre RSRP et RSRQ ? RSRP = puissance d'un RE de signal de référence : couverture, indépendante de la charge. RSRQ = N·RSRP/RSSI : part de la cellule dans tout ce qui est reçu, donc sensible à l'interférence et à la charge. Même RSRP, RSRQ très différent selon l'interférence.

5. Pourquoi le RSRP est-il défini par élément de ressource ? Pour être indépendant de la largeur de bande et de la charge : on compare des cellules de configurations différentes sur une base commune. Conséquence dans le projet : 46 dBm sur 600 sous-porteuses → 18,2 dBm par RE. La v3 l'oubliait (RSRP trop élevés de 27,8 dB).

6. Quelle est la plage de valeurs du RSRP ? De −140 à −44 dBm (TS 36.133). En pratique, au-dessus de −80 c'est excellent, en dessous de −110 c'est le bord de couverture.

7. Qu'est-ce que l'événement A3 ? La voisine devient meilleure que la cellule de service d'un décalage (offset + hystérésis + CIO), pendant au moins le TTT. C'est le déclencheur standard du handover intra-fréquence. Dans le projet : marge de 3 dB, RSRP filtré L3.

8. À quoi servent l'hystérésis et le TTT ? À éviter les déclenchements sur des fluctuations passagères, donc les ping-pongs. Mais un TTT ou une hystérésis trop grands rendent le handover tardif : c'est le compromis de la mobilité.

9. Pourquoi un TTT plus court pour les UE rapides ? Ils traversent la zone de recouvrement en quelques secondes : avec un TTT long, ils sortent de la couverture de la source avant que le handover parte. La norme prévoit le *speed-dependent scaling* ; le projet utilise TTT = max(1, 3 − ⌊v/7⌋) s.

10. Qu'est-ce que le filtrage L3 ? Une moyenne exponentielle des mesures dans la couche RRC : F = (1−a)·F + a·M, avec a = 1/2^(k/4). Avec k = 4 (défaut), a = 0,5, la valeur du projet. Il lisse le fast fading avant la comparaison aux seuils.

11. Citez les types d'échec de mobilité. Trop tardif (RLF avant/pendant le HO), trop précoce (RLF juste après, retour vers la source), mauvaise cellule (rétablissement dans une 3e cellule), ping-pong (aller-retour rapide). Le MRO les compte pour ajuster CIO, hystérésis et TTT.

12. Qu'est-ce qu'une RLF ? *Radio Link Failure* : après N310 indications de perte de synchronisation, le temporisateur T310 démarre ; à son expiration, l'UE déclare la RLF et tente un *RRC Re-establishment*. Dans le projet : 3 s de RLF puis rétablissement sur la meilleure cellule.

13. Pourquoi le SINR chute-t-il avant un handover ? En approchant de la frontière, le signal utile baisse et celui de la voisine (qui est de l'interférence, même fréquence) monte : double effet. C'est aussi pourquoi un handover tardif échoue : l'ordre de handover ne passe plus.

14. Pourquoi le modèle 3GPP UMa ? Il représente un macro-réseau urbain (antennes en toiture, ISD ~500 m) et c'est le modèle de référence des études 3GPP (TR 36.873, puis 38.901 en 5G). Formules différentes en LOS et NLOS.

15. Quelles sont les trois échelles de variation du canal ? Path loss (distance, centaines de m), masquage log-normal (obstacles, ~100 m, σ 4–6 dB), fast fading (multi-trajets, λ/2 = 7,5 cm à 2 GHz). Le terminal moyenne le fast fading sur sa période de mesure.

16. Qu'est-ce que le CQI ? Un indice de 1 à 15 envoyé par l'UE, qui indique la modulation et le codage les plus efficaces décodables avec moins de 10 % d'erreurs de bloc. L'eNodeB s'en sert pour l'adaptation de lien. C'est un SINR quantifié.

17. En quoi votre approche se rapproche-t-elle de la 5G / O-RAN ? La 5G NR utilise les mêmes mesures (SS-RSRP, SS-RSRQ, SS-SINR) et les mêmes événements A3/A5. O-RAN prévoit des applications d'IA (xApps) dans un RIC quasi temps réel, proche des stations de base : c'est exactement l'endroit où un prédicteur de handover se déploierait.

11.4 Questions ML

18. Pourquoi une classification et pas une régression du « temps avant handover » ? La décision opérationnelle est binaire (préparer ou non), et le temps avant le prochain handover est mal défini quand aucun n'arrive (censure). Une formulation de survie ou de régression serait une extension possible.

19. Pourquoi 3 secondes ? Assez pour agir (préparer la cible, avancer le HO) et assez court pour que les signes radio soient visibles. À 10 s, la mobilité devient imprévisible et le label trop bruité.

20. Comment avez-vous géré le déséquilibre (6 % de positifs) ? Pondération des classes (class_weight, scale_pos_weight, sampler équilibré pour les LSTM), une seule par modèle. Puis seuil choisi sur la validation et métriques adaptées (F1, PR-AUC). J'ai d'ailleurs corrigé une double pondération dans les LSTM, qui faisait chuter leur précision.

21. Pourquoi pas de SMOTE ? Le suréchantillonnage synthétique interpole entre exemples : sur des séries temporelles avec des variables de lag, il crée des trajectoires physiquement incohérentes. La pondération atteint le même objectif sans fabriquer de données.

22. Comment avez-vous évité les fuites de données ? Fenêtres glissantes décalées d'un pas ; split temporel, pas aléatoire ; scaler appris sur le train seulement ; aucune variable inobservable (position, LOS) ; seuils et champion choisis sur la validation ; un test automatique vérifie que modifier le futur ne change pas le passé.

23. Pourquoi un split temporel ? Deux secondes consécutives sont presque identiques. Un split aléatoire mettrait l'une en train et l'autre en test : le modèle « reconnaîtrait » le test, et les scores seraient gonflés.

24. Pourquoi ne pas utiliser la position GPS, qui serait très informative ? Le réseau ne la connaît pas en temps réel (vie privée, batterie, précision en intérieur). L'utiliser rendrait le modèle non déployable. La v4 l'a retirée, pour un coût mesuré d'environ 0,01 de F1 (sur la v3).

25. Pourquoi PR-AUC plutôt que ROC-AUC ? Avec des positifs rares, la ROC-AUC reste élevée même avec beaucoup de fausses alarmes (le dénominateur des FPR, ce sont les négatifs, très nombreux). La PR-AUC mesure la précision des alarmes, que l'opérateur paie. Son niveau de hasard vaut le taux de positifs.

26. Pourquoi choisir le seuil sur la validation ? Les modèles pondérés ne sont pas calibrés : 0,5 n'a pas de sens. Choisir le seuil sur le test serait une fuite : le score de test deviendrait optimiste.

27. Pourquoi XGBoost gagne-t-il ? Est-ce significatif ? Il a la meilleure AUC et PR-AUC de validation. Mais les modèles appris sont à quelques millièmes les uns des autres : je ne surinterprète pas le classement. Le message solide est l'écart avec la règle A3.

28. Pourquoi les LSTM ne font-ils pas mieux que les arbres ? Les variables de lag, delta et fenêtre glissante donnent déjà la dynamique aux modèles tabulaires. La séquence brute de 10 s n'ajoute pas grand-chose. Les arbres sont aussi plus robustes avec peu de données et des variables hétérogènes.

29. Bagging vs boosting ? Le bagging (forêt aléatoire) entraîne des arbres indépendants sur des échantillons bootstrap et moyenne : il réduit la variance. Le boosting (XGBoost) ajoute des arbres séquentiellement, chacun corrigeant le gradient des erreurs précédentes : il réduit le biais, avec un learning rate et un arrêt anticipé contre le sur-apprentissage.

30. Pourquoi l'arrêt anticipé sur la PR-AUC ? Sur la log-loss, XGBoost continuait jusqu'à ~900 arbres et sa PR-AUC de validation baissait. La log-loss est dominée par les 94 % de négatifs ; la PR-AUC mesure ce qui compte. Avec elle, il s'arrête vers 120 arbres et la PR-AUC monte à 0,47.

31. Comment fonctionne votre stacking, et quel piège avez-vous évité ? Une régression logistique combine les probabilités de XGBoost, RF et LSTM, entraînée sur la validation. Pour la comparer équitablement aux autres modèles sur cette même validation, j'utilise des prédictions hors échantillon (validation croisée en 5 blocs temporels) ; sinon son score serait optimiste.

32. Qu'est-ce que SHAP, et que montre-t-il ici ? Une décomposition additive de chaque prédiction en contributions des variables (valeurs de Shapley). Pour XGBoost : la vitesse (17,7 %), l'écart service/voisine

`rsrp_diff` (14,9 %), le SINR et le RSRQ dominent ; les variables temporelles pèsent ~37 % au total. La vitesse sert d'a priori (les véhicules changent de cellule 4 fois plus souvent) ; `rsrp_diff` reproduit la logique d'A3.

33. Vos métriques « événementielles », pourquoi ? Un opérateur raisonne en handovers anticipés et en coût des alarmes, pas en secondes bien classées. Je mesure le rappel des handovers à 3 s, le délai d'anticipation et le temps de fausse alarme, et je compare les modèles à budget de fausses alarmes égal.

34. Le F1 de 0,5 n'est-il pas faible ? Le label est intrinsèquement bruité : prédire 3 s à l'avance un événement en partie aléatoire (masquage, transitions LOS). L'AUC de 0,92 montre un bon classement ; en événements, 89 % des handovers sont anticipés. Et le gain sur la règle (+0,10 de F1, PR-AUC ×1,6) est le vrai indicateur de valeur.

11.5 Questions sur la démarche

35. Comment savez-vous que la prédiction améliore le réseau ? Je l'ai mise dans la boucle du simulateur, avec les mêmes trajectoires pour toutes les politiques et un témoin aléatoire à taux d'intervention égal. Résultat : −30 % d'échecs là où le témoin ne fait rien. Mais un TTT court fait presque aussi bien, et les ping-pongs augmentent : c'est la limite actuelle.

36. Quelle est la principale limite du projet ? Les données sont simulées : la vraie validation serait des traces opérateur. Ensuite : pas de temps d'1 s, transitions LOS franches qui gonflent les handovers piétons, réseau de 4 cellules. Et la boucle fermée montre qu'il faut aussi prédire les ping-pongs.

37. Que feriez-vous avec plus de temps ? Prédire les ping-pongs (ou la qualité du handover) en plus du handover ; ré-entraîner sous la politique prédictive ; une approche par apprentissage par renforcement pour régler TTT/CIO ; valider sur des traces réelles ; prédire aussi la cellule cible.

38. Racontez une difficulté rencontrée. En revoyant le projet, j'ai trouvé que `predict.py` rejetait 100 % des lignes d'un fichier brut, et que le dashboard affichait les risques sur les mauvaises lignes quand le fichier était trié par temps. J'ai unifié l'inférence dans un seul module, rattaché les scores par index, et écrit un test qui mélange l'ordre des lignes. Autre exemple : le RSRP calculé avec la puissance totale au lieu de la puissance par RE.

39. Comment garantissez-vous la reproductibilité ? Graines fixées partout (simulateur, scikit-learn, XGBoost, PyTorch, sampler) ; pipeline DVC ; rapports JSON versionnés ; CI qui relance tout. Deux exécutions complètes donnent des métriques identiques.

40. Pourquoi avoir écrit des tests sur un projet de data science ? Parce que les erreurs les plus graves sont silencieuses : une fuite de données ou un décalage de lignes donnent quand même des chiffres, juste faux. Les tests figent les propriétés critiques (pas de fuite, parité en ligne/hors ligne, ordre des lignes sans effet, physique cohérente), et plusieurs reproduisent des bugs réels de la v3.

11.6 Glossaire

Terme	Signification
3GPP	organisme de normalisation des réseaux mobiles (normes TS / TR)
A3, A4, A5	événements de mesure déclenchant un rapport (voisine > service + offset, etc.)
CIO	*Cell Individual Offset* : décalage par voisine pour avancer/retarder les HO
CQI	*Channel Quality Indicator*, 1 à 15
CRS	*Cell-specific Reference Signal*, sur lequel on mesure le RSRP
eNodeB	station de base LTE
EPC / E-UTRAN	cœur de réseau / réseau d'accès LTE
Handover (HO)	transfert intercellulaire
ISD	*Inter-Site Distance*
L1 / L3	couche physique / couche RRC (filtrage des mesures)
LOS / NLOS	avec / sans trajet direct
MME, S-GW, P-GW, HSS	entités du cœur EPC
MRO	*Mobility Robustness Optimisation* (fonction SON)
OFDMA	accès multiple par répartition en fréquences orthogonales
PR-AUC	aire sous la courbe précision-rappel (*average precision*)
RB / RE	*Resource Block* (12 sous-porteuses × 1 slot) / *Resource Element*
RLF	*Radio Link Failure*
RRC	*Radio Resource Control* (couche 3 radio)
RSRP / RSRQ / RSSI	puissance de référence / qualité / puissance totale reçue
SHAP	méthode d'explicabilité fondée sur les valeurs de Shapley
SINR	rapport signal sur interférence plus bruit
SON	*Self-Organizing Networks*
TTT	*Time-To-Trigger*
UE	*User Equipment* (terminal)
UMa	*Urban Macro* (scénario de propagation 3GPP)
X2 / S1	interface eNodeB–eNodeB / eNodeB–cœur